

UNIVERSITAT POLITÈCNICA DE VALÈNCIA

Universitat Politècnica de València

Departamento de Sistemas Informáticos y Computación

DSIC DEPARTAMENTO DE SISTEMAS INFORMÁTICOS Y COMPUTACIÓN

Máster Universitario en Ingeniería y Tecnología de Sistemas Software

Trabajo Fin de Máster

Tipos Cociente en un Lenguaje Dependientemente Tipado

Autor(a): Luis Vegas
Director(a): Santiago Escobar

Valencia, 2026

Este Trabajo Fin de Máster se ha depositado en el Departamento de Sistemas Informàticos y Computación de la Universitat Politècnica de València para su defensa.

Trabajo Fin de Máster
Máster Universitario en Ingeniería y Tecnología de Sistemas Software

Título: Tipos Cociente en un Lenguaje Dependientemente Tipado

Septiembre, 2026

Autor(a): Luis Vegas
Director(a): Santiago Escobar
Departamento de Sistemas Informáticos y Computación
Universitat Politècnica de València

Tabla de contenidos

1. Introducción **1**
1.1. Motivación 1
1.2. Objetivos 1
1.3. Estructura de la tesis 1

2. Preliminares **3**
2.1. El cálculo lambda 3
2.1.1. Booleanos 4
2.1.2. β-reducción 4
2.1.3. Variables libres y ligadas 4
2.1.4. Sustitución 5
2.1.5. Confluencia 6
2.2. Teoría de Tipos 8
2.2.1. Tipado à la Curry y à la Church 16
2.2.2. Lambda Cálculo Simplemente Tipado 16
2.3. Tipos dependientes 17
2.3.1. Tipos sigma 20
2.4. Tipos cociente 21

3. Estado del arte **25**
3.1. Cocientes en teoría de tipos intensional 25
3.2. Nuprl y la teoría de tipos extensional 26
3.3. El cociente en Lean 26
3.4. Tipos inductivos superiores y teoría cubical 26
3.5. Formalización de sistemas de tipos en lógica de reescritura 26

4. Maude y la lógica de reescritura **29**
4.1. Sorts y Lógica Ecuacional Ordenada 29
4.2. Módulos Funcionales 30
4.3. Módulos de Sistema y Reescritura no Determinista 31
4.4. Formalización de Lenguajes y Sistemas de Tipos 31
4.5. Reflexión y meta-programación 33

5. Un Lenguaje Dependientemente Tipado en Maude **35**
5.1. Sintaxis 35
5.2. Sustitución 36
5.3. Normalización 37
5.4. Verificación de Tipos 38

6. Extensión con Tipos Cociente **41**
6.1. Introducción 42
6.2. Formación 42
6.3. Eliminación y Computación 43

7. Casos de uso y validación **45**
7.1. Los enteros como pares de naturales 45
7.2. Multiconjuntos como listas módulo permutación 45
7.3. Los enteros módulo n 45
7.4. Propiedades de la especificación 45

8. Conclusiones, limitaciones y trabajo futuro **47**
8.1. Conclusiones 47
8.2. Limitaciones 47
8.3. Trabajo futuro 47

A. Código Maude **49**

Capítulo 1

Introducción

1.1. Motivación

1.2. Objetivos

1.3. Estructura de la tesis

Capítulo 2

Preliminares

2.1. El cálculo lambda

El cálculo lambda fue inventado por Alonzo Church en la década de los 80 con el propósito de capturar la noción abstracta de la aplicabilidad de una función, sin embargo, conviene precisar en qué sentido. Una función puede entenderse de dos maneras: de forma extensional, como el conjunto de pares entrada–salida que la determina, o de forma intensional, como la regla o el proceso de cómputo que, dado un argumento, prescribe cómo obtener el resultado (Geuvers, 2014). Es esta segunda lectura, la de la función como algoritmo, la que el lambda cálculo formaliza (Barendregt, 1984; Geuvers, 2014). En su origen, Church concibió el sistema como parte de una fundamentación de la lógica y las matemáticas; dicha formulación resultó ser inconsistente, tal y como evidenciaron Kleene y Rosser, por lo que Church aisló su núcleo puramente computacional, libre de constantes lógicas, dando lugar al cálculo lambda tal y como se conoce hoy (Barendregt, 1984), siendo ampliamente adoptado como un sistema formal de lógica matemática.

Este sistema ocupa, además, un lugar central en el diseño de lenguajes de programación. De forma análoga, la manera en la que se programa en el mismo es muy similar a cómo se programa en un lenguaje funcional hoy en día. De hecho, lenguajes como Haskell, Agda, la familia de lenguajes basados en ML, y Lisp, están ampliamente inspirados en un conjunto del cálculo lambda y, pueden entenderse, en buena medida, como extensiones dotadas de azúcar sintáctica, sistemas de tipos y otras primitivas (Pierce, 2002).

El cálculo lambda es un lenguaje muy potente y, a su vez, sumamente simple. Un término puede estar compuesto, o bien, formarse, de tres elementos: variables, aplicación y abstracción lambda. De forma inductiva, fijado un conjunto infinito numerable de variables V, el conjunto Λ de los términos lambda se define como el menor conjunto cerrado bajo las siguientes reglas (Barendregt, 1984; Pierce, 2002):

$$
\begin{aligned}
M := \quad & x && (1)\\
\mid \quad & \lambda x.\, M && (2)\\
\mid \quad & M\ N && (3)
\end{aligned}
$$

Donde, (1) es un conjunto infinito de variables, tal que si $x, y, z \in V$ y $V \in \Lambda$, entonces

$V \in \Lambda$. (2) es una abstracción lambda, si $x \in V$ y además $M \in \Lambda$, entonces $\lambda x.\, M \in \Lambda$. (3) Es una aplicación de dos términos, si $M, N \in \Lambda$ entonces $MN \in \Lambda$.

El lambda cálculo es lo suficientemente potente para permitirnos codificar simples tipos de datos o, al menos, emular el comportamiento que obtendríamos de los mismos en un sistema de tipos más avanzado.

2.1.1. Booleanos

El tipo booleano es un tipo habitado por solo dos posibles términos: $True$ o $False$. Pese a que el cálculo lambda sin tipos no posee una noción de verdad o falsedad, es posible, aún así, codificar los booleanos (Barendregt, 1984):

Ejemplo 2.1. $\text{True} \equiv \lambda xy.\, x, \quad \text{False} \equiv \lambda xy.\, y$

A continuación se detallan propiedades de sumo interés que articulan, posteriormente, los fundamentos del lenguaje que describe esta tésis: la distinción entre variables libres y ligadas, junto con la α-equivalencia que de ella se deriva; la operación de sustitución que evita la captura de variables; y la β-reducción como noción primaria de cómputo. Se mencionará igualmente la η-conversión, que captura la extensionalidad de las funciones.

2.1.2. β-reducción

La noción de evaluacion de un término viene dada por beta-reducción, donde la notación $\to_\beta$ es empleada. La $\to_\beta$ está basada en la *sustitución*, formalmente expresada como *M[x := N]*, significando que *M* es un término donde en su cuerpo, *x*, ha sido reemplazado por *N*.

Por tanto, una expresión de la forma $(\lambda x.\, M)\, N$ puede ser reescrita como *M[x := N]*, donde toda ocurrencia de *x* en *M* (estando *x* libre en *M*), debe ser reemplazada por *N* (Geuvers, 2014).

Ejemplo 2.2. $(\lambda x.\, x + 1)\, 5$ reduce a $(x + 1)[x := 5] = 5 + 1 = 6$.

Ejemplo 2.3. La función identidad aplicada a un término cualquiera devuelve dicho término: $(\lambda x.\, x)\, y \to_\beta x[x := y] = y$.

Ejemplo 2.4. La β-reducción puede encadenarse, reduciendo paso a paso hasta que no queden más redexes. considérese el combinador $K \equiv \lambda x.\, \lambda y.\, x$ aplicado a dos argumentos:

$$\begin{aligned}(\lambda x.\, \lambda y.\, x)\, a\, b &\to_\beta (\lambda y.\, x)[x := a]\, b \\ &= (\lambda y.\, a)\, b \\ &\to_\beta a[y := b] = a.\end{aligned}$$

Nótese que el segundo argumento, b, es descartado, pues y no ocurre en el del lambda.

2.1.3. Variables libres y ligadas

La abstracción $\lambda x.\, M$ introduce un *binder* sobre la variable x, cuyo *alcance* es el cuerpo de M. Decimos, pues, que una ocurrencia de una variable x está *ligada* cuando aparece dentro del alcance del λx que la introduce; en caso contrario, dicha ocurrencia se dice *libre*.

Conviene resaltar que una misma variable puede ocurrir libre y ligada dentro de un mismo término. En $(\lambda x.\, x)\, x$, por ejemplo, la primera ocurrencia de x está ligada por el λx, mientras que la segunda, externa a la abstracción, permanece libre. Formalmente, el conjunto de variables libres de un término, denotado $FV(M)$, se define de forma inductiva sobre la estructura del término (Pierce, 2002):

$$
\begin{aligned}
&(1)\ FV(x) = \{x\},\\
&(2)\ FV(\lambda x.\, M) = FV(M) \setminus \{x\},\\
&(3)\ FV(M\, N) = FV(M) \cup FV(N).
\end{aligned}
$$

(1) indica que una variable, por sí sola, es libre. (2) establece que al abstraer sobre x, esta deja de ser libre en $\lambda x.\, M$, pues queda ligada por el nuevo binder; por ello se retira del conjunto. (3) recoge que las variables libres de una aplicación son, simplemente, la unión de las libres de cada subtérmino M y N respectivamente. Un término M tal que $FV(M) = \emptyset$ se denomina *cerrado*.

Por último, el nombre concreto de una variable ligada carece de relevancia: lo único que importa es la estructura de ligaduras que establece. Así, $\lambda x.\, x$ y $\lambda y.\, y$ denotan exactamente la misma función identidad. A esta noción, que identifica términos que difieren únicamente en el nombre de sus variables ligadas, se le conoce como α-*equivalencia*, y se denota $=_\alpha$. El proceso de renombrar una variable ligada para obtener un término α-equivalente recibe el nombre de α-*conversión*, y será fundamental para definir correctamente la sustitución bajo un binder.

2.1.4. Sustitución

Anteriormente, definimos la sustitución en términos de $\to_\beta$ reducción, sin embargo, la sustitución posee en sí reglas de computo que valen la pena considerar pues, dependiendo del término que se esté evaluando, la sustitución tendrá, o no, efecto alguno.

Se definen, pues, las reglas de computo de la sustitución de acuerdo a lo discutido por Pierce (2002). Más formalmente:

$$
\begin{aligned}
&(1)\ x[x := N] \equiv N,\\
&(2)\ y[x := N] \equiv y \text{ if } x \not\equiv y,\\
&(3)\ (PQ)[x := N] \equiv (P[x := N])(Q[x := N]),\\
&(4)\ (\lambda y.\, P)[x := N] \equiv \lambda z.\, (P^{y\to z}[x := N]),\ \text{si } \lambda z.\, P^{y\to z} \text{ es un } \alpha\text{-variante de}\\
&\quad \lambda y.\, P \text{ tal que } z \notin FV(N).
\end{aligned}
$$

Para (1), sustituyendo x, por N dentro de x, siendo x el término que buscamos sustituir, obtenemos N. De forma análoga, sustituyendo cada ocurrencia de x en el término y, por N, si x e y no son iguales, el término intacto, y, será retornado. El caso de (3), tiene sentido cuando P es una abstracción aplicada a un término Q. Concretamente, se reduce a la sustitución de x por N, en ambos P y Q. (4) contempla la sustitución bajo un binder. Lo que se busca es sustituir x por N dentro del cuerpo

P de la abstracción $\lambda y.\, P$. Sin embargo, si procediéramos de forma ingenua y N contuviera a y como variable libre, dicha y quedaría equivocadamente atrapada por el binder λy, alterando por completo el significado del término. A este fenómeno se le conoce como *captura de variables*.

Para evitarlo, antes de sustituir se renombra la variable ligada y por una variable fresca z, mediante α-conversión, escogida de modo que $z \notin FV(N)$ (y, por extensión, que no genere conflicto con las variables ya presentes en P). Solo entonces se procede a sustituir x por N dentro del cuerpo ya renombrado. De esta manera, la sustitución preserva el significado del término original, razón por la cual se la describe como una sustitución *que evita la captura* (Pierce, 2002).

Ejemplo 2.5. Supóngase la sustitución $(\lambda y.\, x)[x := y]$. Si sustituyéramos directamente, obtendríamos $\lambda y.\, y$, es decir, la identidad: la y libre que introdujimos habría sido capturada por el binder. Renombrando primero la variable ligada a una fresca z, obtenemos $(\lambda z.\, x)[x := y] = \lambda z.\, y$, que es el resultado correcto, pues la y permanece libre.

2.1.5. Confluencia

Anteriormente señalamos cómo la β-reducción emula, en cierto sentido, el proceso de cómputo de una función dada: evaluado un término, aspiramos a obtener su forma normal, esto es, un término en el que ya no resta redex alguno que nos permita seguir reduciendo (Barendregt, 1984; Pierce, 2002).

Ahora bien, el orden en que se contraen tales redexes no es indiferente, pues puede influir en si el cómputo termina o, mejor dicho, en si alcanza una forma normal. Surge así una pregunta: dado un término t del que divergen dos reducciones, $t \to_\beta^* t_1$ y $t \to_\beta^* t_2$, en uno o varios pasos, ¿es posible reconciliar ambas ramas en un término común u alcanzable tanto desde t_1 como desde t_2? Cuando tal u existe para cualquier par de reducciones divergentes, decimos que la reducción es *confluente* sobre t.

En el presente apartado describimos con detenimiento la propiedad de confluencia y, exponémos también, por qué la β-reducción la satisface.

La confluencia, conocida también como teorema de Church-Rosser, es una propiedad de los sistemas de reescritura que caracteriza cuándo dos términos, derivados de un término inicial, por reducción en uno o más pasos, convergen a un mismo resultado. Su relevancia radica en que el resultado del cómputo queda determinado con independencia del orden en que se ejecuten los pasos de reducción (Barendregt, 1984).

Teorema 2.1 (Church-Rosser)**.** *Sea M un término del λ-cálculo. Si $M \to_\beta^* N_1$ y $M \to_\beta^* N_2$, entonces existe un término N_3 tal que $N_1 \to_\beta^* N_3$ y $N_2 \to_\beta^* N_3$ (Barendregt, 1984, p. 62).*

Conviene precisar que el teorema habla de la *clausura reflexivo-transitiva* de la reducción, $\to_\beta^*$, es decir, de cero o más pasos de β-reducción. Es importante resaltar que la β-reducción no cumple la versión más fuerte de la propiedad, conocida como propiedad del diamante en un solo paso: si de M contraemos dos redexes distintos en un único paso para obtener N_1 y N_2, no siempre existe un N_3 alcanzable desde

ambos en un solo paso. La razón es la duplicación, pues un redex puede copiar a otro cuando el argumento contiene, a su vez, redexes, de modo que un lado de la bifurcación necesita varios pasos para reabsorber las copias que el otro lado ya había contraído. Lo que el teorema de Church-Rosser garantiza es la versión global, sobre $\rightarrow_\beta^*$, que sí se sostiene.

Ademas, por simplicidad, hasta ahora hemos de forma ligera abusado del término de confluencia. Resulta importante separar dos nociones de la misma: la confluencia local y la confluencia global. Decimos que un sistema es *localmente confluente* cuando, partiendo de M y dando un paso hacia N_1 y otro hacia N_2, existe un término común alcanzable desde ambos en cero o máas pasos. La confluencia local es más débil que la confluencia global, y la implicación de la primera a la segunda no siempre se cumple, pues solo se obtiene bajo hipótesis adicionales, como la terminación, por tanto, una relacion (sistema?) terminante es confluente, si es localmente confluente. de acuerdo al lema de Newman. (Baader, 2012; Barendregt, 1984).

Lema 2.2. *1. M es una forma normal si M no contiene más redexes.*

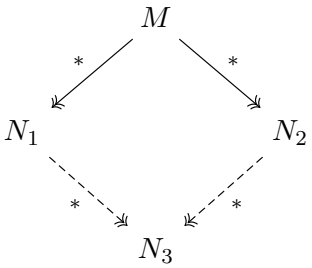
2. M tiene una forma normal si existe un N, en forma normal, tal que $M =_\beta N$. Tal N es la forma normal de M.

Una consecuencia inmediata, y quizá la más útil en la práctica, es la unicidad de la forma normal. Si un término admite forma normal, esta es única salvo α-equivalencia.

Corolario 2.3. *Si M posee una forma normal, esta es única salvo α-equivalencia. En efecto, supóngase que N_1 y N_2 son ambas formas normales de M, de modo que $M \rightarrow_\beta^* N_1$ y $M \rightarrow_\beta^* N_2$. Por el teorema de Church-Rosser existe N_3 con $N_1 \rightarrow_\beta^* N_3$ y $N_2 \rightarrow_\beta^* N_3$. Pero N_1 y N_2 están en forma normal, luego no contienen redex alguno y no pueden reducirse en pasos no triviales, de donde $N_1 =_\alpha N_3 =_\alpha N_2$.*

Este corolario, por un lado, justifica que hablemos de *la* forma normal de un término, en singular, cuando esta existe. Por otro, garantiza que el valor de un cómputo, entendido como su forma normal, queda determinado por el término de partida y no por la estrategia de evaluación escogida, de modo que el orden en que decidamos contraer los redexes solo puede afectar a si terminamos, mas no al resultado al que llegamos.

La confluencia se representa mediante el siguiente diagrama, cuya forma de rombo explica que la propiedad reciba también el nombre de *propiedad del diamante*. El vértice superior es el término de partida M, las dos diagonales descendentes son las reducciones divergentes hacia los términos N_1 y N_2, y las diagonales punteadas representan los pasos que aún restan, garantizados por el teorema, para reconciliar ambas ramas en un término común N_3.



Ejemplo 2.6. Considérese el término $M \equiv (\lambda x.\, x\, x)\, ((\lambda y.\, y)\, z)$, que presenta dos redexes: el externo, que aplica $\lambda x.\, x\, x$ a su argumento, y el interno, $(\lambda y.\, y)\, z$. Contrayendo primero el redex externo, el argumento se duplica y obtenemos $N_1 \equiv ((\lambda y.\, y)\, z)\, ((\lambda y.\, y)\, z)$, que aún requiere dos pasos para resolver las dos copias del redex interno. Contrayendo primero el redex interno, en cambio, llegamos en un solo paso a $N_2 \equiv (\lambda x.\, x\, x)\, z$. Ambas ramas convergen en $z\, z$:

Obsérvese cómo la rama izquierda necesita más de un paso para alcanzar la confluencia, lo cual ilustra por qué el teorema se enuncia sobre $\rightarrow_\beta^*$ y no sobre la reducción en un único paso.

De aquí emerge un problema que tendrá mención a lo largo del trabajo: la terminación. La terminación determina si el cómputo de un término concluye o no. En el λ-cálculo, una secuencia de reducciones puede no detenerse, entrando en un ciclo infinito de pasos que impide alcanzar forma normal alguna, como ilustra el siguiente ejemplo.

Ejemplo 2.7. El ejemplo canónico de término sin forma normal es el combinador $\Omega \equiv (\lambda x.\, x\, x)\, (\lambda x.\, x\, x)$. Al contraer su único redex, el argumento $\lambda x.\, x\, x$ se sustituye en el cuerpo $x\, x$, devolviendo de nuevo Ω:

$$(\lambda x.\, x\, x)\, (\lambda x.\, x\, x) \rightarrow_\beta (x\, x)[x := \lambda x.\, x\, x] = (\lambda x.\, x\, x)\, (\lambda x.\, x\, x) \equiv \Omega.$$

La reducción se cierra pues, sobre sí misma:

$$\Omega \equiv (\lambda x.\, x\, x)\, \overset{\beta}{\underset{\leftarrow}{(\lambda x.\, x\, x)}}$$

El término es trivialmente confluente, ya que su única reducción conduce siempre a sí mismo, y sin embargo carece de forma normal. Este ejemplo evidencia que la confluencia y la terminación son propiedades independientes: un sistema puede ser confluente sin ser terminante.

El ejemplo de Ω pone de manifiesto que el λ-cálculo sin tipos no es terminante, pues existen términos que carecen de una forma normal y cuya reducción no se detiene en ningún momento. La terminación, entendida como normalización fuerte, se recupera al imponer una disciplina de tipos sobre los términos. En particular, el λ-cálculo simplemente tipado sí es fuertemente normalizante, propiedad que abordarémos en un capítulo posterior, al tratar dicho sistema.(Geuvers, 2014; Pierce, 2002).

2.2. Teoría de Tipos

La teoria de tipos es una rama de las matemáticas y la lógica que tiene como objeto formalizar no solo términos matemáticos, tales como una variable x o una función

cualquiera f y las operaciones entre ambas, si no también, la idea de que cada término forma parte o tiene un tipo concreto, es decir, que el tipo de una variable $x : \mathbb{N}$ en el dominio de los numeros naturales, es completamente distinguible al tipo de una función $f : \mathbb{N} \to \mathbb{N}$.

Los tipos, pese a que son similares a los conjuntos, son ligeramente distintos. Por conjunto, entiéndase como una colección de elementos, similares o no, que poseen una propiedad concreta:

$$C = \{rojo, amarillo, azul\} \tag{2.1}$$

de donde es posible extenderlos y razonar sobre más relaciones de pertenencia y formar, por ejemplo, conjuntos dentro de otros conjuntos y definir que un elemento pertenece a varios conjuntos a la vez:

$$C' = \{\text{verde}, \text{naranja}\} \cup C \tag{2.2}$$

Cosa que, en los tipos, no es posible, y es que un término es de (o pertenece a) un tipo único. Otra característica que los distingue, es que los conjuntos dan información semántica y, los tipos por otro lado, proveen información sintáctica de los objetos. Por ejemplo, comunmente en la teoría de conjuntos definimos una relación de pertenencia como sigue:

$$4 \in \{\, n \in \mathbb{N} | x > 5 \,\} \tag{2.3}$$

En la teoría de tipos, un juicio de tipado, sigue la siguiente notación que, a lo largo de la presente tesis estarémos empleando:

$$3 + 1 : \mathbb{N} \tag{2.4}$$

Esto puede interpretarse, desde el lente de teoría de tipos, como $3 + 1$ es un término en los naturales, o bien, de tipo $\mathbb{N}$. Aún así, las mismas operaciones aplicadas en la teoría de conjuntos son trasladables a la teoría de tipos. Tómese como ejemplo la unión disjunta de conjuntos, definida formalmente como la unión disjunta $A \sqcup B$ de dos conjuntos, es un conjunto A indexado por B:

$$\bigcup_{i \in I} \{(x, i) : x \in A_i\} \tag{2.5}$$

Esta operacion resulta en el tipo de dato algebraico $\tau_1 + \tau_2$, donde el valor concreto del tipo puede bien ser la proyeccion del lado izquiero o el derecho.

La noción de tipos se atribuye a Bertrand Russell, quien desarrolló la teoría de tipos en respuesta a las contradicciones que, a comienzos del siglo XX, aquejaban a los fundamentos de la matemática. El contexto es el del programa logicista y la teoría de conjuntos naciente. Hacia finales del siglo XIX, Georg Cantor había desarrollado la teoría de conjuntos como un marco general en el que toda la matemática podía, en principio, expresarse, y Gottlob Frege perseguía un objetivo todavía más ambicioso en sus *Grundgesetze der Arithmetik*: reducir la aritmética a la lógica pura. Ambos proyectos descansaban, de forma más o menos explícita, sobre el principio de comprensión irrestricta, según el cual toda propiedad bien definida determina un

conjunto, el de todos los objetos que la satisfacen (Coquand, 2022).

Russell advirtió en 1901 que ese principio conducía a una contradicción. Si toda propiedad define un conjunto, entonces la propiedad de no pertenecer a sí mismo define el conjunto de todos los conjuntos que no son miembros de sí mismos:

$$R = \{ x \mid x \notin x \} \tag{2.6}$$

Preguntarse si R pertenece o no a sí mismo lleva, por tanto, a una contradicción lógica, pues por la propia definición de R se tiene que:

$$R \in R \iff R \notin R \tag{2.7}$$

es decir, R es miembro de sí mismo exactamente cuando no lo es.

Lo que Russell sugeria pues, es que estas paradojas comparten una misma estructura, todas involucran una definición que se refiere a una totalidad de la que el objeto definido forma parte. A esto lo llamó el principio del círculo vicioso, que prohíbe que una entidad se defina en términos de una colección a la que ella misma pertenece. La expresión $x \in x$, sobre la que se construye R, es precisamente una de estas definiciones circulares, o autoreferencia.

Para evitarlas, Russell propuso organizar los objetos en una jerarquía estratificada. En el nivel más bajo están los individuos; en el siguiente, los predicados (o clases) de individuos; después los predicados de predicados, y así sucesivamente. Esta distinción es la que la teoría de tipos internaliza: a cada objeto se le asigna un tipo, y una función proposicional solo puede aplicarse a argumentos del tipo inmediatamente inferior. Bajo esta restricción, la fórmula $x \in x$ ni siquiera llega a estar bien y es rechazada, ya que la pertenencia exige que su argumento sea de un tipo menor que el del conjunto. El conjunto R no puede construirse, y la paradoja desaparece por construcción. De este modo, conjuntos grandes y pequeños residen en niveles distintos de la teoría.

La formulación original de Russell, desarrollada junto a Whitehead en *Principia Mathematica*, era la teoría ramificada de tipos, que sumaba a la estratificación por tipos una segunda por órdenes para neutralizar también las paradojas semánticas, al precio de requerir el discutido axioma de reducibilidad. Ramsey observó más tarde que las paradojas estrictamente lógicas o conjuntistas quedan resueltas ya con la jerarquía simple de tipos, sin necesidad de los órdenes ni del axioma de reducibilidad. Es esta teoría simple de tipos la que, años después, Church retomaría como base de su lambda cálculo tipado, y la que enlaza directamente con el objeto de este trabajo.

En términos más generales, los tipos son ubicuos en computación, y numerosos lenguajes tienen su propio sistema de tipos, unos mas avanzados que otros y, otros más simples. Tan simples que, la manipulación de tipos es casi perceptible o inexitstente. Lenguajes con esta capacidad de omitir tipos son conocidos como lenguajes dinámicamente tipados, donde en casos, es completamente opcional tipar expresiones y en caso contrario, el compilador hace lo mejor que puede para inferir un tipo adecuado para una expresión. Haskell por ejemplo, es un lenguaje de alto nivel, ampliamente utilizado por grupos de investigación y académicos, su sistema de tipos esta inicialmente basado en el cálculo lambda simplemente tipado, también llamado *System F*,

que abordaremos en capítulos posteriores a este. Sin embargo, hoy en día su sistema de tipos ha madurado lo suficiente como para dejar de considerarle un lenguaje con un sistema de tipos simple. C++ es un claro ejemplo de un sistema de tipos, quizá, mucho más complejo, pero sumamente rico, que facilita posteriormente, una distincion semántica y rechaza programas que son imposibles de evaluar correctamente. El lenguaje en el que estarémos trabajando en el presente trabajo, Maude, es de alto nivel pero fuertemente tipado. Aunque precisamente no utiliza tipos internamente, si no Sorts, la manipulación de términos y el enunciar si un término es de tipo (o sort) A es, en principio, lo mismo. OCaml en principio, fue de los lenguajes que llego a explorar más a fondo la teoria de tipos y la expresividad de los mismos como funcionalidades del lenguaje, siendo de los primeros en implementar tipos polimórficos, es decir, tipos que pueden tomar la forma de otros tipos. Finalmente, un sistema de tipos permite al lenguaje alcanzar capacidades de expresividad bastante amplias y, a su vez, evitar que programas erronoes se ejecuten.

Dentro de los problemas que intenta resolver la teoria de tipos, no se encuentra solo encontrar un tipo para un término concreto, sino también, determinar (si lo hay) un término para un tipo y si un término esta bien tipado (o es un tipo legal) dentro de un contexto (Geuvers, 2014). Listamos pues, algunos de los problemas que se abordan en la teoria de tipos:

1) Tipabilidad: este problema formula la pregunta: ¿existe un tipo bien tipado para el término en concreto en el contexto?

$$\Gamma \vdash a :?$$

No siempre existe un tipo bien definido para un término. Las derivaciones o juicios de tipado sirven justamente para determinar logicamente cual tipo es el adecuado o, correctamente, inferir el tipo del término. No encontrar dicho tipo, por supuesto, ocasionaría un error de tipado.

2) Type-checking: determina si el tipo de un término en un contexto Γ cualquiera, es correcto o bien tipado.

$$\Gamma \overset{?}{\vdash} a : A$$

Sea $\Gamma = \{b : B, U : *\}$, si A no es un tipo bien formado en Γ, el sistema de tipos lo rechaza. Esto es claro, pues, no hay una regla que permita derivar que $a : A$ esta bien formado en Γ.

3) Habitabilidad: el problema de la habilitabilidad (también llamado term finding), formula la pregunta: ¿existe un término, dado un tipo bien definido y un contexto concreto?

$$\Gamma \overset{?}{\vdash} ? : A$$

Nótese que los problemas anteriormente mencionados son decibibles en $\lambda \rightarrow$, es decir, para cada una de estas cuestiones existe un algoritmo posible que produce un

resultado en el lambda cálculo simplemente tipado. En sistemas de tipos más complicados, sin embargo, la búsqueda de un término llega a ser un problema, pues tales sistemas no logran encontrar un habitante del tipo en el contexto, siendo problemas no decidibles (Barendregt et al., 2013).

Se empleará, a lo largo de la tesis, la siguiente notación para definir las derivaciones en la teoría de tipos, donde la parte superior es una lista de premisas y la parte inferior, la conclusión o los consecuentes:

$$\frac{premisa_1 ... \quad premisa_n}{consecuente_1 ... consecuente_n} \tag{2.8}$$

Para $n \geq 0$. Las reglas pueden leerse bien de arriba hacia abajo (como *teniendo que, podemos concluir que*) o de abajo hacia arriba (como *dado que se cumple el consecuente, las premisas deben cumplirse*). Adicionalmente, las variables presentes en premisas y consecuentes, estan universialmente quantificadas a menos de que se indique lo contrario. Por tanto,

$$\frac{}{\forall x, P(x)} \tag{2.9}$$

Donde $\forall$ es implícito.

$$\frac{\Gamma \vdash A \quad \Gamma, a : A}{\Gamma \vdash a : A} \tag{2.10}$$

Que puede leerse como: *si, dado que en el contexto* Γ*,* A *es un tipo bien formado y, además, en el mismo contexto* Γ *extendido con* $a : A$*, entonces podemos inferir que* a *es de tipo* A *en* Γ.

Dentro de la teoría de tipos existen seis tipos de juicios que serán de utilidad en la medida en que se estudiarán los sistemas formales en este trabajo:

$\vdash \Gamma \ ctx$	Γ es un contexto valido
$\Gamma \vdash \sigma \ Type$	σ es un tipo
$\Gamma \vdash M : \sigma$	M es un término de tipo σ en Γ
$\vdash \Gamma \equiv \Delta$	$\Gamma \ y \ \Delta$ son contextos definicionalmente iguales
$\Gamma \vdash \sigma \equiv \tau \ Type$	$\sigma \ y \ \tau$ son tipos definicionalmente iguales en Γ
$\Gamma \vdash M \equiv N : \sigma$	$M \ y \ N$ son términos definicionalmente iguales, de tipo σ, en Γ

Adicionalmente, independientemente del sistema de tipos que se describa, las mismas reglas estructurales se adoptan, como bien describe Hofmann, 2009.

$$\frac{}{\vdash \emptyset}\ \text{C-Emp} \qquad \frac{\Gamma \vdash \sigma}{\vdash, \Gamma\ x : \sigma}\ \text{C-Ext} \tag{2.11}$$

$$\frac{\vdash \Gamma = \Delta \quad \Gamma \vdash \sigma = \tau\ \mathsf{Type}}{\Gamma, x : \sigma \equiv \Delta,\ y : \tau}\ \text{C-Eq} \tag{2.12}$$

$$\frac{\vdash \Gamma, x : \sigma}{\Gamma \vdash x : \sigma}\ \text{Var} \tag{2.13}$$

Asimismo, para las reglas que expresan la igualdad definicional como una relación de equivalencia, tenemos que:

$$\frac{\vdash \Gamma\ ctxt}{\vdash \Gamma = \Gamma\ ctxt}\ \text{C-Eq-R} \tag{2.14}$$

$$\frac{\vdash \Gamma = \Delta\ ctxt}{\vdash \Delta = \Gamma\ ctxt}\ \text{C-Eq-S} \tag{2.15}$$

$$\frac{\vdash \Gamma = \Delta\ ctxt \quad \vdash \Delta = \Theta\ ctxt}{\vdash \Gamma = \Theta\ ctxt}\ \text{C-Eq-T} \tag{2.16}$$

$$\frac{\Gamma \vdash \sigma\ \mathsf{type}}{\Gamma \vdash \sigma = \sigma\ \mathsf{type}}\ \text{Ty-Eq-R} \tag{2.17}$$

$$\frac{\Gamma \vdash \sigma = \tau\ \mathsf{type}}{\Gamma \vdash \tau = \sigma\ \mathsf{type}}\ \text{Ty-Eq-S} \tag{2.18}$$

$$\frac{\Gamma \vdash \sigma = \tau\ \mathsf{type} \quad \Gamma \vdash \tau = \rho\ \mathsf{type}}{\Gamma \vdash \sigma = \rho\ \mathsf{type}}\ \text{Ty-Eq-T} \tag{2.19}$$

$$\frac{\Gamma \vdash M : \sigma}{\Gamma \vdash M = M : \sigma}\ \text{Tm-Eq-R} \tag{2.20}$$

$$\frac{\Gamma \vdash M = N : \sigma}{\Gamma \vdash N = M : \sigma}\ \text{Tm-Eq-S} \tag{2.21}$$

$$\frac{\Gamma \vdash M = N : \sigma \quad \Gamma \vdash N = O : \sigma}{\Gamma \vdash M = O : \sigma}\ \text{Tm-Eq-T} \tag{2.22}$$

$$\frac{\Gamma \vdash M : \sigma \quad \vdash \Gamma = \Delta\ ctxt \quad \Gamma \vdash \sigma = \tau\ \mathsf{type}}{\Delta \vdash M : \tau}\ \text{Tm-Conv} \tag{2.23}$$

$$\frac{\vdash \Gamma = \Delta\ ctxt \quad \Gamma \vdash \sigma\ \mathsf{type}}{\Delta \vdash \sigma\ \mathsf{type}}\ \text{Ty-Conv} \tag{2.24}$$

A lo largo de los años numerosos esfuerzos se han realizado para desarrollar distintas extensiones de la teoría de tipos y poder dar suficiente poder expresivo al lambda cálculo. Estos sistemas de tipos fueron explorados por H. Barendregt en el Cubo Lambda (Barendregt, 1991), para investigar las diferentes dimensiones en las que

puede extenderse el sistema formal. En su paper describe como cada dimension del cubo representa una dependencia entre términos y tipos, y sus posibles combinaciones.

Entiéndase por dependencia como la dependencia mutua, valga la redundancia, que existe entre tipos y términos: términos que dependen de otros términos, términos dependientes de tipos, tipos dependientes de términos, y tipos dependientes de otros tipos. Los ocho vertices del cubo, consecuencia de combinar cada posible dimensión, resultan en diferentes sistemas de tipos y niveles de expresividad. Concretamente, abordaremos en el presente trabajo, la extension del vertice inferior derecho, que corresponde a la extension de tipos dependientes, concreamente $\lambda\Pi$, o refiérase también en la literatura como λP.

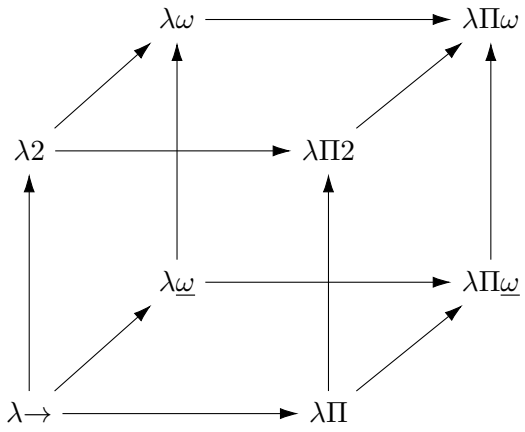


Figura 2.1: El cubo lambda de Barendregt

la dependencia que define cada vertice, es descrita por la siguiente tabla:

Sistema	Conjunto de reglas			
$\lambda\!\rightarrow$	$(*,*)$			
$\lambda 2$	$(*,*)$	$(\Box,*)$		
$\lambda\mathrm{P}$	$(*,*)$		$(*,\Box)$	
$\lambda\mathrm{P}2$	$(*,*)$	$(\Box,*)$	$(*,\Box)$	
$\lambda\underline{\omega}$	$(*,*)$			$(\Box,\Box)$
$\lambda\omega$	$(*,*)$	$(\Box,*)$		$(\Box,\Box)$
$\lambda\mathrm{P}\underline{\omega}$	$(*,*)$		$(*,\Box)$	$(\Box,\Box)$
$\lambda\mathrm{P}\omega = \lambda\mathrm{C}$	$(*,*)$	$(\Box,*)$	$(*,\Box)$	$(\Box,\Box)$

Es necesaria una explicación a los conjuntos de reglas más relevantes y nivel de expresividad que permite cada sistema. $\lambda\!\rightarrow$ es posiblemente el más común. Formalmente este sistema es conocido como el lambda cálculo simplemente tipado que se abordará con más dedicación en un capítulo aparte, donde para construir una abstracción lambda, basta con proveer otro término. Por tanto FM es, por ejemplo, un término en Λ donde F depende de M. La notación $(*,*)$ indica precisamente en qué nivel del sistema de tipos viven los términos para poder formarse.

Ejemplo 2.8.

$$(\lambda x : \mathbb{N}.t)3 \quad \longrightarrow_\beta \quad 3 \tag{2.25}$$

$\lambda 2$ es conocido en la literatura como *System F*. Es el lambda cálculo de segundo orden, o polimórfico. Fue descubierto por Girard y Reynolds en 1972 en el cual se introduce el cuantificador universal en el cálculo y poder cuantificar sobre tipos. A parte de ello, a partir de aquí algunas implementaciones extienden el sistema introduciendo kinds en la teoría predicativa del lenguaje. Un gran número de lenguajes de programación funcional, como Haskell, están inspirados en este sistema. Además, en $\lambda 2$ se presenta la noción de variables de tipos, con las cuales es posible cuantificar sobre un rango de tipos, permitiendo así, la suficiente expresividad al lambda cálculo para definir, por instancia, la función identidad polimórfica.

Ejemplo 2.9.

$$(\Lambda\alpha\lambda x : \alpha.x)\, 3 \longrightarrow_\beta 3$$
$$(\Lambda\alpha\lambda x : \alpha.x)\, \mathit{true} \longrightarrow_\beta \mathit{true}$$

Donde Λ representa el cuantificador universal sobre los tipos del lenguaje.

$\lambda\underline{\omega}$ añade operadores a nivel de tipos, o bien, funciones que toman uno o varios tipos y devuelven otro. Para ello se introduce un sistema de *kinds* que distingue los tipos propios, de kind $\star$, de los constructores de tipos, de kind $\star \Rightarrow \star$. Un tipo puede entonces depender de otros mediante abstracción y aplicación a nivel de tipos, del mismo modo que un término depende de otros a nivel de valores.

Ejemplo 2.10. El constructor de pares toma dos tipos y devuelve el tipo de la tupla.

$$\mathit{Pair} \;\equiv\; \lambda\alpha : \star.\ \lambda\beta : \star.\ \alpha \times \beta$$

Así, Pair Nat Bool reduce a Nat $\times$ Bool.

$\lambda\omega$ combina los operadores de tipos anteriormente presentados y el polimorfismo de $\lambda 2$, así, ahora es posible cuantificar universalmente sobre tipos y constructores de tipos.

Ejemplo 2.11. Como ejemplo tómese la función $swap$ que intercambia la posición de los elementos en una tupla. Cuantificando ambos argumentos α y β, podemos aplicar $swap$ sobre cualquier tipo que habite $\star$.

$$\mathit{swap} \;:\; \forall\alpha : \star.\ \forall\beta : \star.\ \mathit{Pair}\ \alpha\ \beta \to \mathit{Pair}\ \beta\ \alpha$$

λP representa los tipos que dependen de términos. Los tipos dependientes, como se les referirá a lo largo de este etrabajo, extienden el sistema de tipos con familias de tipos, o tipos indexados por los elementos de otros tipos. La siguiente regla de inferencia formaliza esta idea donde x puede o no, aparecer en B. Observese que de no aparecer x en B, estariamos involucrándonos con el tipo de función $A \to B$.

$$\frac{\Gamma \vdash A : *,\ x : A \quad \Gamma \vdash B : A}{\Gamma \vdash \Pi x : A.Bx} \tag{2.26}$$

Tómese como ejemplo *Vec A n* donde la longitud n del vector, depende de cierto parámetro. Decimos, por tanto, que *Vec A n* es un tipo dependiente o bien una familia de tipos indexado por n. Asimismo, la linea con la que se distinguen términos y tipos se hace más difusa y, en consecuencia, ambos comparten el mismo espacio semántico. Por ello, encontramos la constante $\star$ que representa el *sort* en el que habitan todos los tipos, por tanto, la siguiente derivación es posible en λP: $A : \star \vdash (A \to A) : \star$, que expresa que *dado que A es un tipo bien formado, (A → A), es entonces, un tipo* (Barendregt, 1991). El símbolo $\Box$ no es propiamente un término en la sintaxis, en cambio, es un símbolo en la meta-teoría para afirmar que se tiene una derivación de kinds válida y predicativa, por tanto, se puede entender $\Gamma \vdash (\kappa : \Box)$ mediante la siguiente lectura: *k es un kind bien formado dado el contexto* Γ.

Combinando todos los vértices obtenémos λC o *Calculus of Constructions* donde el lambda cálculo alcanza un nivel expresivo bastante avanzado y los sistemas del cubo cohabitan en uno solo. Lenguajes como Coq y Agda, están basados en λC. Este es un sistema de tipos puro bajo el paradigma de proposiciones como tipos, con tipos dependientes, jerarquias de universos de tipos y un tipo Prop que distingue de $\star$.

2.2.1. Tipado à la Curry y à la Church

En el lenguaje que se formaliza en el presente trabajo, cada variable lleva consigo su tipo. Pese a que garantiza decibilidad en cada expresión que se construya, esta decisión no es arbitraria. Describe Reynolds (2000) que existen dos formas en las que es posible entender qué significa que una expresión tenga un tipo. En la semántica intrínseca solo los términos (a los que Reynols se refiere como frases) guardan un sentido si satisfacen un juicio de tipado y, por otro lado, una expresion carece de sentido si no satisface ningun juicio de tipado y, de satisfacer varios, poseerá tantos significados como juicios.

El tipado à la Church, que es la forma que adoptarémos, deja en evidencia de forma explícita el tipo al que pertenece una variable ligada, por otro lado, el tipado à la Curry omite las anotaciones y el chequeo para determinar su tipo es delegado a otro proceso de búsqueda o inferencia. La ventaja que se manifiesta en el tipado à la Church es que la verificación del tipo en Maude viene dada por pattern matching y, de manera ecuacional, restringimos que cada término bien formado admita un único tipo o pertenezca a un tipo en concreto.

Considérese como ejemplo la expresión $\lambda x : int.x$ su significado intrínseco es la función identidad sobre los enteros. La identidad sobre los booleanos, por otro lado, que se denota como $\lambda x : bool.x$ es una función distinta y, por tanto, un término distinto. Es decir, el tipo determina el significado del término. Por el contrario $\lambda x.x$, su significado extrínseco es la función identidad sobre el universo de valores que pudiesen aparecer en un cómputo, por tanto, los tipos $bool \to bool$, $int \to int$ se le atribuyen de forma externa, como predicados que dicho significado satisface.

2.2.2. Lambda Cálculo Simplemente Tipado

Añadir tipos al lambda cálculo resuelve el problema de la terminación que en section 2.1.5 estudiamos, permitiendo la creación y evaluación de términos en el cálculo

mucho más restrictivo, esto es, anotando cada término con un tipo previamente definido que nos brindará un poco más de información sobre el dominio o tipo de valor que esperamos retornar. Este sistema es también denotado como $\lambda \rightarrow$ y en definitiva, tiene como objeto evitar anomalías que permitan la construcción de programas no terminantes o no lógicamente plausibles Geuvers (2014).

La forma canónica de implementar tipos en λ es, en principio, definir un conjunto infinito de variables $\mathbb{V} = \{\alpha, \beta, \gamma\}$ y un conjunto $\mathbb{T}$ de tipos: $\tau, \tau \rightarrow \tau$. Los miembros de este conjunto estan definidos como:

1. Variable de tipo: donde si $\alpha \in \mathbb{V}$ entonces $\alpha \in \mathbb{T}$
2. Tipo función: donde si $\alpha, \tau \in \mathbb{T}$ entonces $(\alpha \rightarrow \tau) \in \mathbb{T}$

La gramática para el lambda calculo simplemente tipado, se extiende, como sigue:

Definición 2.1. Extensión a $\lambda \rightarrow$

$$M = \ldots \mid x : \tau \mid \lambda x : \tau . e$$
$$\mathbb{T} = \{\tau \mid \tau \rightarrow \tau\}$$

Los tipos en $\lambda \rightarrow$ nos permiten enunciar juicios de la forma $M : \tau$, que se entiende como *el término M es de tipo τ*, asumiendo que $M \in \Lambda$ y τ es un tipo bien definido. A continuación, se explican algunos detalles a tomar en cuenta para cada término que, desde ahora, llevan consigo un tipo.

1. Aplicación: para cubrir el caso donde $M \cdot N$, es menester conocer la signatura de M. Sea M y N términos en Λ, donde M es de tipo $\sigma \rightarrow \tau$ y N de tipo σ, es claro observar que N posee el mismo tipo que el del dominio de M por tanto, es asertado confirmar que el tipo de la aplicación es de tipo $M \cdot N : \tau$.
2. abstracción: cuando $\lambda x : \tau . M$ donde $M : \gamma$, podemos asertar que es una abstraccion con codominio en γ y dominio en τ.

2.3. Tipos dependientes

Los tipos dependientes son, en principio, tipos que dependen de términos. Conforman parte del trabajo de Per Martin-Löf y su teoría de tipos intuisionista (Martin-Löf, 1975) que tiene como objeto extender la interpretación de proposiciones como tipos en la lógica de predicados, como estarémos estudiando en este capítulo. Antes de adentrarnos en la teoría de tipos dependientes, es necesario dedicar de forma breve una introducción a los sistemas de tipos puros, framework de donde parte esta formalización lógica. Un PTS es un framework para especificar y formalizar el lambda cálculo y extender su teoría. Fue propuesto inicialmente por Berardi y Terlow como una generalizacion del cubo lambda de Barendregt (1991). A diferencia del lambda cálculo tradicional, un PTS dispone de pseudo-términos que heredan, en gran parte, sintáxis del λ cálculo simplemente tipado que hasta ahora conocemos.

El conjunto de pseudo-términos, se define como sigue:

Definición 2.2. Pseudo-términos de un PTS

$$\mathcal{T} = \mathcal{T} \mid \mathcal{T}\,\mathcal{T} \mid \mathcal{S} \mid \lambda \mathcal{V} : \mathcal{T} . \mathcal{T} \mid \Pi \mathcal{V} : \mathcal{T} . \mathcal{T}$$

Asimismo, resaltan Barthe and Coquand algunas características a tomar en cuenta que ofrecen una imágen más amplia de la definición de un sistema de tipos puro.

1. Se utiliza la misma convención para extraer el conjunto de variables libres en un pseudo-término $FV(M)$. Adicional a ello, se escribirá $A \to B$ para todo tipo de función en el que x no aparezca libre en B. Y se usará $\Pi x : A.B(x)$, en caso contrario.

2. Los sistemas de tipos puros, describen un λ cálculo a la Church, es decir, las abstracciones lambda llevan consigo el dominio de las variables ligadas y, como consecuencia de ello, la computabilidad en un sistema de tipos puro es decidible.

3. Los sistemas de tipos puros proveen los cimientos de los tipos dependientes, de hecho, es en el constructor del tipo Π donde se evidencia, por primera vez, la dependiencia en esta teoría, capturando la noción del producto cartesiano generalizado.

La noción de tipo dependiente es análoga al producto cartesiano dependiente, o generalizado de la teoría de conjuntos. En los conjuntos es posible, para cada conjunto A, y una familia de conjuntos $B(x)_{x \in A}$ indexados por A, formar un nuevo conjunto $\Pi_{x \in A} B_x$, donde sus elementos son funciones con dominio en A, tal que $f(a) \in B_a$ para todo $a \in A$. Consideramos, entonces, $\Pi_{x \in A} B_x$ un producto cartesiano dependiente, pues B_a en el cual $f(a)$ vive, depende de a. De igual forma opera el tipo Π que hemos mencionado en el conjunto de pseudo-términos que conforman un PTS: sea A un tipo definido en Γ y $B(x)$ un tipo, donde x es una variale de tipo A, decimos pues, que $\Pi x : A.B(x)$ es un tipo dependiente donde para todo x en A, existe una familia de tipos $B(x)$.

Definición 2.3. Especificación. La especificación de un PTS es una tripla $\mathcal{S} = (\mathcal{S}, \mathcal{A}, \mathcal{R})$, donde:

1. $\mathcal{S}$ es un conjunto de sorts.

2. $\mathcal{A} \subseteq \mathcal{S} \times \mathcal{S}$ es un conjunto de axiomas.

3. $\mathcal{R} \subseteq \mathcal{S} \times \mathcal{S} \times \mathcal{S}$ es un conjunto de reglas.

Además, se presentan las reglas de formación de un PTS, según lo sugiere Barthe and Coquand:

Nótese que, a diferencia de los sistemas de tipos estudiados hasta ahora, existe una ligera disparidad en el contexto presente entre el lambda cálculo simplemente tipado y uno dependientemente tipado, y es que, primero, los contextos ahora pueden tener presencia de variables de tipos, de la forma $A : s$ con $s : \mathcal{S}$. Segundo, el orden en el que se insertan juicios $A : s$ en Γ importa para poder manejar de manera correcta y consistente las dependencias, por ejemplo, un juicio de la forma $A : s, \sigma : A$ es totalmente válido, pero $\sigma : A, A : s$ carece de sentido, pues no existe un juicio previo a $\sigma : A$ del que sea posible derivar que A es un tipo bien formado.

Como se ha mencionado ya con anterioridad, la extensión a la que se dedicará el estudio del presente trabajo sera λP, sin embargo, se invita al lector a revisar con más detalles el resto de teorías que, con bastante precisión Barthe and Coquand

abordan sobre el Cubo Lambda de Barendregt (Barendregt, 1991).

Definición 2.4. Sea $S = \{\star, \Box\}$ de sorts y el conjunto $\mathcal{A} = \{(\star : \Box)\}$ de axiomas. Es posible obtener λP extendiendo el conjunto de reglas, de forma que $\mathcal{R} = \{(\star, \star), (\star, \Box)\}$. Permitiendo λP introducir los axiomas $\{\star, \Box\}$ es posible definir los tipos dependientes, por tanto $B : \star$ podria contener una subexpresión $M : A : \star$.

La importancia o interés de estudio de los tipos dependientes, radica en cómo estos habilitan al entendimiento de la correspondencia entre las proposiciones como tipos y los programas como pruebas a la luz de la lógica de predicados. El conjunto de reglas definidas en 2.4 nos muestra que es posible formar expresiones que habitan en $*$ y así obtener como resultado una expresión que, también, habita en $*$. De forma intuitiva, esto corresponde a una función ordinaria del tipo $\mathbb{N} \to \mathbb{N} : *$, donde el dominio es de sort *****, y el codominio de sort *****. Adicional a ello, el segundo miembro de la tupla, nos indica que podemos formar una expresión que habita en $*$ y como resultado, obtenemos una que habita en $\Box$. Esto último corresponde a la posibilidad de derivaciones como $\Pi x : A.B(x)$ cuando $A : *$ y $B : \Box$, siendo B un tipo indexado por A. Por tanto, la siguiente derivación es posible:

$$\frac{\Gamma \vdash A : * \quad \Gamma \vdash n : \mathbb{N}}{\Gamma \vdash \text{Vec } A\ n : *}\ \text{Vec-Form} \tag{2.27}$$

Donde *Vec A n* $: *$ habita en $\Box$.

Dado que a los tipos pudiesen contener términos, una noción de igualdad definicional es necesaria en la teoría, por ejemplo, conviene considerar los términos $0 : \mathbb{N}$ y $0+0 : \mathbb{N}$ como definicionalmente iguales y, por tanto, no habría distinción entre *Vec(0)* y *Vec 0 + 0*. Si *M : Vec(0)* y *N: Vec 0 + 0*, nos lleva a un juicio de la forma $M = N : \sigma$ que se lee *M y N son términos definicionalmente iguales en el tipo sigma*, y, además, que $\sigma = \tau : A$, que se lee *sigma y tau son tipos definicionalmente iguales en A*.

Definición 2.5 (Igualdad definicional)**.** Decimos que dos términos son definicionalmente iguales cuando reducen, luego de cero o más pasos de reducción, a un término en común Mimram (2020). La igualdad a la que hacemos referencia en este caso, no es igualdad proposicional, denotada con $\equiv$, si no, a la igualdad que es inherente del lenguaje base $=$ que corresponde a la equivalencia de términos bajo β-reducción.

```
1  op _+_ : Nat Nat -> Nat .
2  eq 0 + N = N .
3  eq s(N) + M = s(N + M) .
```

En este tipo de igualdad el lenguaje no puede descirnir de dos términos, pues son exactamente iguales por definición. Por ejemplo, es posible asertar que *0 + N* y *N* son definicionalmente iguales dada la definición inductiva de la suma que en Maude, podemos proponer que *0 + N* reduce a *N* en un solo paso, por tanto, es posible usar *0 + N* y *N* de forma intercambiable a lo largo del programa.

Sin embargo, esto no siempre es el caso: en contraste *n + 0*, que no es definicionalmente igual a *n* porque *no existe* una ecuación en nuestra definición de la suma

que inmediatamente pueda asertar que, en este caso, *0 + n* sí reduce a *n*. Pero sí que podemos demostrar proposicionalmente, mediante una prueba más elaborada por inducción en n, de que esta igualdad sí se cumple y, finalmente, concluir que *n + 0* $\equiv$ *n*.

Definición 2.6 (Igualdad proposicional)**.** Decimos que dos términos son proposicionalmente iguales cuando existe un término p que habita $\mathrm{Id}_A(a, b)$, por tanto, existe una prueba tal que $p : Id(a, b)$ es un testigo de esta igualdad en A. A diferencia de la igualdad definicional, que se obtiene en términos de β-reducción y es una igualdad inherente del lenguaje, la igualdad proposicional debe ser demostrada explícitamente mediante la construcción de un testigo, esto es, un término p que es prueba de dicha igualdad. El constructor canónico del tipo identidad es $\mathrm{refl}_a : \mathrm{Id}_A(a, a)$, que atestigua que todo término es trivialmente igual a sí mismo. Para demostrar igualdades no triviales, se requiere construir un término del tipo correspondiente, tipicamente, por inducción.

Ejemplo 2.12. Considerando el mismo ejemplo de la suma sobre naturales, definida por recursion sobre el primer argumento, la proposicion $0 + n \equiv n$ es una igualdad definicional, pues el término $0 + n$ reduce directamente a n por la primera clausula, de modo que refl_n basta como testigo. En cambio, la proposicion $n + 0 \equiv n$ no es una igualdad definicional, ya que cuando n es una variable, la expresion $n + 0$ no puede reducirse porque Maude no encuentra una ecuacion que inmediatamente pueda cubrir este caso. Es necesario construir el testigo por inducción sobre n:

- El caso base, donde si $n = 0$, entonces $0 + 0$ reduce a 0, y refl_0 es el testigo.
- El caso inductivo, donde si $n \coloneqq \mathrm{suc}(k)$, entonces $\mathrm{suc}(k) + 0$ reduce a $\mathrm{suc}(k + 0)$ por la segunda clausula. Aplicando $\mathrm{cong}_{\mathrm{suc}}$ a la hipótesis inductiva $p_k : k+0 \equiv k$, obtenemos el testigo $\mathrm{cong}_{\mathrm{suc}}(p_k) : \mathrm{suc}(k + 0) \equiv \mathrm{suc}(k)$.

En Maude, podemos simular este comportameinto de la siguiente manera, asumiendo que el tipo de proposición es un sort cualquiera.

```
1  sort A B .
2  subsort A B < Nat .
3  sort Proof .
4  op cong : (A -> B) Proof -> Proof .
5  op refl : -> Proof.
6
7  op zero' : Nat -> Proof .
8  eq zero'(0) = refl .
9  eq zero'(s(N)) = cong(zero'(N))
```

La existencia de esta prueba es lo que nos permite concluir que $n + 0 \equiv n$ proposicionalmente.

Corolario 2.4. *Dos términos que son definicionalmente iguales, también lo son proposicionalmente. Lo contrario no tiene por qué cumplirse.*

2.3.1. Tipos sigma

Los sistemas de tipos puros, en principio, no proveen constructores más allá del tipo Π, por tanto, el próximo constructor o tipo al que dedicarémos un breve acercamiento, que también formará parte de la teoria y lenguaje que se formalizará en un capitulo

posterior, es el tipo sigma. Representado en el lambda cálculo simplemente tipado como $A \times B$, es el tipo de una tupla en el que el primer y segundo componente son de tipo A y B respecivamente, sin embargo, en los tipos dependientes, el segundo componente B pudiese tener una ocurrencia x de tipo A, luego, su tipo, tiene la signatura: $\Sigma x : A.B$, o bien $\langle M, N \rangle_{\Sigma x:A}$, donde M habita en A y N en $B(N)$. Para facilitar o, más bien, asegurar la decibiliad del prototipo, este tipo estará acompañado de su adscripción.

El conjunto de pseudo-términos T, se extiende, por tanto, con un nuevo miembro representando los tipos sigma:

$$T = ... \mid \Pi V : T.T \mid \langle T, T \rangle \mid \mathit{fst}\, T \mid \mathit{snd}\, T$$

Donde *fst* y *snd* son *proyecciones* que extraen el primer y segundo componente de la tupa $\langle T, T \rangle$. Sus reglas de formación y eliminación, plantea (Hofmann, 2009), se definen como sigue:

$$\frac{\Gamma \vdash A : * \quad \Gamma \vdash n : \mathbb{N}}{\Gamma \vdash \mathrm{Vec}\ A\ n : *}\ \Sigma\text{-Form} \tag{2.28}$$

$$\frac{\Gamma \vdash A : * \quad \Gamma \vdash n : \mathbb{N}}{\Gamma \vdash \mathrm{Vec}\ A\ n : *}\ \Sigma\text{-Form} \tag{2.29}$$

Los tipos sigma serán utiles para probar propiedades del lenguaje en la práctica y, con suerte, comprobar que la formalización de la teoria de tipos estudiada en este trabajo cumple con su función.

2.4. Tipos cociente

Los cociente son un término ubicuo en matemáticas, sin embargo, desde la teoría de conjuntos están presentes y se extiende dicho concepto con naturalidad a la teoría de tipos. En los conjuntos, es posible definir el cociente de un conjunto S en clases de equivalencia, obteniendo el cociente del conjunto, donde cada elemento que lo conforma es considerado igual, o equivalente, dada una relacion de equivalencia $\sim$. Así mismo, es posible definir el cociente de un tipo, donde dada una relación de equivalencia, los elementos son iguales dentro del cociente pero, fuera de ellos, aún es posible distinguir uno de otro. Los tipo cociente nos permiten redefinir, entonces, la igualdad proposicional en un sistema de tipos.

Los tipos cociente son el concepto clave del presente trabajo, puesto a que son sumamente importantes en la formalización de conceptos matemáticos y el razonamiento de programas de lenguajes funcionales. Los cocientes, son un concepto de la teoría de tipos extensional. Las alternativas que la teoría intensional ofrece para simularlos se discuten en chapter 3.

Definición 2.7. Una relación de equivalencia $\sim$ en un conjunto A es una relación binaria de signatura $R : A \to A \to Prop$ que cumple las siguientes propiedades:

1. Reflexividad: $\forall a \in A,\ a \sim a$.
2. Simetría: $\forall a, b \in A,\ a \sim b \to b \sim a$.
3. Transitividad: $\forall a, b, c \in A,\ a \sim b \to b \sim c \to a \sim c$.

Por tanto, la clase de equivalencia de un elemento $a \in A$ se denota $[a]$, donde $[a] = \{x \in A \mid a \sim x\}$. El conjunto de clases de equivalencia de A viene denotada como A/R, que se lee *el cociente de A por R*. De forma intuitiva, un tipo cociente A/R es un tipo donde su igualdad ha sido redefinida por una relación de equivalencia $\sim$.

Es necesario mencionar la utilidad practica de los cocientes y, dejar claro que no son un concepto puramente matemtico. Tómese como ejemplo el caso de los naturales representados como un par (a, b), donde con su diferencia $a - b$ permite identificar que muchos de ellos pueden representarse comeo un mismo entero, donde, $(3, 1), (4, 2) y (5, 3)$ representaria el numero 2 y ser, de alguna forma, indistiguibles para el programa. La condicion que los agrupa, $a + b = b + c$ es en este caso, la relacion de equivalencia, formar el cociente dada esta relacion es lo que nos permite que colapsen estos elementos en un solo valor, $[t]$, conocido como *representante*, que es indiscernible por el sistema de tipos.

Los multiconjuntos o Set siguen el mismo patrón. Recuérdese estas como listas en las que el orden y la repetición de los elementos que la conforman carecen de importancia alguna. Partimos, pues, del tipo de las listas y cocientamos por la relación que identifica dos de ellas cuando una es permutación de la otra. Así $[1, 2, 2]$ y $[2, 1, 2]$ se vuelven el mismo objeto dentro del cociente.

En definitiva, desde el lado de la programación, disponer de cocientes amplía el repertorio de tipos de datos algebraicos y facilita razonar sobre tipos dependientes, nociones de igualdad y sobre la semántica de programas concurrentes, como sugieren Li (2014) y Hofmann (1995).

Según lo estudiado por Li (2014), las reglas de formación de los tipos cociente se definen como sigue:

$$\frac{\Gamma \vdash A : *, \quad \Gamma x, y : A \vdash x \sim y : Prop}{\Gamma \vdash A/\sim} \text{ Q-Form} \tag{2.30}$$

Donde $A/\sim$ es el tipo cociente, dado que A es un tipo bien formado en Γ, x y y habitan en A, y existe una prueba, de tipo $Prop$ que atestigua que $x \sim y$.

Adicionalmente, la regla $Q - intro$ permite inferir que $[a]$ es una clase de equivalencia de tipo $A/\sim$, dado que $a : A$ en Γ

$$\frac{\Gamma \vdash a : A}{\Gamma \vdash [a] : A/\sim} \text{ Q-Intro} \tag{2.31}$$

La regla $Q - Ax$, indica que dada una prueba p de equivalencia de $a \sim b$, y dos elementos a y b de tipo A, nos es posible definir que $[a]$ y $[b]$ son iguales definicionalmente dentro del cociente $A/\sim$, pero, fuera de el, son elementos totalmente distinguibles por el sistema de tipos.

$$\frac{\Gamma \vdash a, b : A \ \Gamma \vdash p : a \sim b}{\Gamma \vdash Qax(p) : [a] =_{A/\sim} [b]} \text{ Q-Ax} \tag{2.32}$$

Además de ello, se propone una regla de eliminación que permite aplicar una función f en $A/\sim$, refereida como *lifting*, si y solo si, la misma respeta la relación de equivalencia (Hofmann, 1995).

$$\frac{\Gamma \vdash B \qquad \Gamma \vdash f : A \to B \qquad \Gamma, a, b : A, p : a \sim b \vdash f^{\sim}(a, b, p) : f(a) =_B f(b) \qquad \Gamma \vdash q : A/\sim}{\Gamma \vdash \hat{f}(q) : B}\ \text{(Q-elim)} \tag{2.33}$$

La regla de cómputo $Q-comp$ indica que al aplicar f (asumiendo que f es una función que respeta la equivalencia) a un representante $[a]$ del cociente, obtenemos, $f(a)$.

$$\frac{\Gamma \vdash a : A}{\Gamma \vdash \text{Qcomp}(a) : \hat{f}([a]) = f(a)}\ \text{Q-comp} \tag{2.34}$$

El resto de reglas estudiadas por Li (2014) se omitiran, pues se consideran fuera del alcance de este trabajo.

Capítulo 3

Estado del arte

3.1. Cocientes en teoría de tipos intensional

En un principio, Martin Löf propuso dos variantes de la teoría de tipos que se diferencian por el tratamiento que dan a la igualdad: la teoria de tipos intencional, y la teoria de tipos extensional. La mayoria de lenguajes de programación de hoy en día, parten de una teoría de tipos intencional porque sus propiedades son decidibles, donde en la teoría de tipos extensional no se garantiza del todo. La razón de esto, es que en la teoría de tipos intensional, la igualdad proposicional definida por tipos de identidad no implica igualdad definicional, y la verificación de tipos es decidible. En la teoría de tipos extensional, por otro lado, la igualdad proposicional se identifica con la igualdad definicional, lo que hace que la verificación de tipos sea indecidible (Li, 2014).

Debido a sus buenas propiedades computacionales, la teoría de tipos intensional es más popular, sin embargo, carece de algunos conceptos extensionales importantes, como la extensionalidad funcional y los tipos cociente, que estaremos abordando.

Muchos asistentes de pruebas y lenguajes con tipos dependientes como Coq, Agda o Lean se apoyan en la teoría intensional. Al ser decidible su igualdad definicional, la verificación de tipos se puede automatizar y todo programa bien tipado es terminante, que son justo las garantías que se espera de un lenguaje con el que se pretende razonar de forma fiable o, mejor dicho, sensata. Lo que se paga a cambio es la ausencia de los conceptos extensionales que venimos comentando: la extensionalidad funcional, la identificación de pruebas distintas de una misma proposición y los tipos cociente (Li, 2014).

Podría pensarse que basta con postular estos conceptos como axiomas, pero esa vía tiene un coste, y es que un axioma se limita a afirmar que cierto término existe, sin decir cómo se calcula, y esa falta de garantía es lo que imposibilita al sistema de tipos determinar si una determinada prueba de igualdad existe o no. Como consecuencia de esto, pues, obtenemos términos cerrados que ya no reducen a ningún valor concreto, con lo que perdemos la canonicidad y, con ella, la razón de haber elegido la teoría intensional desde el principio.

En la teoria de tipos intensional, como se mencionó, no existe la noción de tipos cociente dada su naturaleza de redefinir la igualdad definicional, sin embargo, los setoides se han implementado para capturar el mismo comportamiento.

Los setoides, sugieren Li (2014) y Hofmann (1995) definidos como $(A, \sim)$, están compuestos por:

1. Un tipo $A : Type$.
2. Una relacion binaria $\sim$: $A \to A \to Prop$.
3. Una prueba p de que una función f respeta la equivalencia (es decir, que es reflexiva, transitiva y simetrica).

Una función $f : A \to B$ respeta la equivalencia $\sim$ si $\forall(x, y : A) \to x \sim y \to f(x) =_B f(y)$.

Sin embargo, en la practica el modelo de los setoides no es para nada practico pues, las pruebas de que una función f respeta una equivalencia $\sim$ debe pasarse a toda función repetidas veces.

La alternativa que ha resultado viable, consiste en dejar intacta la sintaxis y buscar interpretaciones donde los tipos sean algo más que conjuntos. En esa línea aparecieron durante los años noventa varios modelos, entre ellos los modelos de setoides de Hofmann y de Altenkirch, y el modelo de grupoides de Hofmann y Streicher (Hofmann, 1995). Leer los tipos como grupoides abrió una puerta que otros siguieron hacia estructuras de dimensión cada vez mayor, y en esa misma dirección se sitúa la teoría homotópica de tipos propuesta por Voevodsky, donde los tipos se piensan como espacios, los términos como puntos en dichos espacios y los tipos de identidad como caminos entre ellos. Bajo esa lectura la igualdad deja de ser un predicado auxiliar y pasa a ser un tipo con estructura propia en cada nivel. Estos últimos detalles, los abordaremos de forma superficial en la teoria de tipos de Martin Lof para complementar la teoria de tipos extensional y dar una breve introduccion a las ideas de la teoría homotópica de tipos.

3.2. Nuprl y la teoría de tipos extensional

3.3. El cociente en Lean

3.4. Tipos inductivos superiores y teoría cubical

3.5. Formalización de sistemas de tipos en lógica de reescritura

La capacidad de Maude para definir sintaxis con notación definible por el usuario, junto con su soporte para reescritura y para reescritura no determinista, lo convierten en una plataforma especialmente apropiada para formalizar lenguajes de programación y sistemas de tipos. La idea general es trasladar la sintaxis abstracta del lenguaje a una signatura Σ, las reglas de tipado o de evaluación de la semántica operacional a reglas de reescritura R, y las identidades estructurales sobre los términos, como la α-equivalencia o la igualdad definicional, a ecuaciones E. Esta metodología

ha sido aplicada con éxito a lenguajes reales, desde fragmentos de C hasta Java o JavaScript, dando lugar a prototipos no solo ejecutables, sino también analizables con las herramientas del entorno formal de Maude (Durán et al., 2020).

Capítulo 4

Maude y la lógica de reescritura

La reescritura de términos constituye, a la luz de los capítulos anteriores, una generalización natural de la β-reducción del λ-cálculo. Donde antes orientábamos un redex de la forma $(\lambda x.\, M)\, N$ hacia $M[x := N]$, ahora disponemos de un conjunto arbitrario de ecuaciones $l = r$, leídas como reglas $l \to r$, y las aplicamos a un término siempre que en él aparezca una instancia del lado izquierdo. El proceso de cómputo se reduce, entonces, a buscar redexes y a contraerlos, exactamente igual que antes, pero en un marco mucho más flexible que permite especificar tipos de datos, funciones y, como veremos, sistemas concurrentes enteros.

Maude es un lenguaje declarativo de alto rendimiento, diseáado y mantenido a lo largo de más de dos décadas por un consorcio en el que participan SRI International, la Universitat Politécnica de Valéncia, la Universidad Complutense de Madrid, la Universidad de Málaga, la Universidad de Illinois en Urbana-Champaign y la Universidad de Stanford (Durán et al., 2020; Clavel et al., 2007). Su fundamento teórico es la lógica de reescritura, que admite simultáneamente dos lecturas. Por un lado, es un marco semántico en el que pueden especificarse de modo natural una amplia variedad de sistemas concurrentes, desde redes de Petri hasta cálculos de procesos. Por otro, es un marco lógico en el que se pueden codificar otras lógicas tomando sus reglas de inferencia como reglas de reescritura. La máxima que sintetiza esta dualidad, y que articula buena parte del diseño del lenguaje, es la equivalencia

$$\text{Cómputo} \;=\; \text{Deducción}$$

de modo que un programa Maude no es otra cosa que una teoría de reescritura, y la ejecución del programa coincide con la búsqueda de derivaciones en dicha teoría (Clavel et al., 2007). Esta correspondencia es la que hace de Maude un candidato especialmente adecuado para implementar prototipos ejecutables de sistemas de tipos, pues la sintaxis abstracta del lenguaje que se quiera modelar y sus reglas de tipado pueden trasladarse de manera casi directa a Maude.

4.1. Sorts y Lógica Ecuacional Ordenada

La primera diferencia notable respecto del λ-cálculo es que Maude obliga a tipar los términos desde la propia signatura. Cada constante y cada operación declara explícitamente los sorts de sus argumentos y de su resultado, donde por *sort* entendemos

lo que en la teoría de tipos llamaríamos un tipo base. La idea de organizar el universo de discurso en sorts proviene de la lógica many-sorted, una variante de la lógica de primer orden en la que el dominio se descompone en una colección indexada de subdominios y cada símbolo de función o predicado especifica el sort de cada uno de sus argumentos (Manzano and Aranda, 2022). Esta restricción sintáctica es muy poco distinguible, pero realmente nos permite determinar, por ejemplo, los términos aritméticos de los términos booleanos sin necesidad de codificarlos en un dominio común, y descarta en principio expresiones sin sentido como sumar un valor de verdad a un número natural.

Maude va, sin embargo, un paso más allá de la lógica many-sorted habitual y adopta la lógica ecuacional *order-sorted*. En esta variante, los sorts forman un conjunto parcialmente ordenado $(S, \leq)$ en el que la relación $s \leq s'$ se interpreta como inclusión de subsort. La idea es la siguiente: si declaramos que NzNat $\leq$ Nat, todo número natural no nulo es también, automáticamente, un número natural, sin necesidad de coerciones explícitas. Las operaciones pueden además estar sobrecargadas a lo largo de la jerarquía de subsorts, de manera que una misma suma puede declararse a la vez sobre Nat y sobre NzNat, y el sistema seleccionará en cada caso el sort más preciso para el resultado.

4.2. Módulos Funcionales

La unidad básica de especificación en Maude es el módulo. Un módulo funcional, declarado entre las palabras clave `fmod` y `endfm`, agrupa una signatura de sorts y operaciones junto con un conjunto de ecuaciones que se interpretan como reglas de reescritura de izquierda a derecha. Es la unidad análoga, en este lenguaje, a lo que en programación funcional sería un módulo con sus tipos algebraicos y sus funciones puras. Considerese el siguiente ejemplo, en el que especificamos los números naturales en notación de Peano:

```
fmod NAT-PEANO is
  sorts NzNat Nat .
  subsorts NzNat < Nat .
  op 0 : -> Nat [ctor] .
  op s : Nat -> NzNat [ctor] .
  op _+_ : Nat Nat -> Nat .
  vars N M : Nat .
  eq 0 + N = N .
  eq s(N) + M = s(N + M) .
endfm
```

El módulo declara dos sorts, NzNat y Nat, con la inclusión NzNat $\leq$ Nat. La constante 0 se introduce con el atributo `ctor`, que la marca como constructor del tipo, y el sucesor s se declara con el mismo atributo. La suma se especifica con notación infija mediante guiones bajos, y sus dos ecuaciones recogen, casi literalmente, la definición estándar por inducción sobre el primer argumento. Una vez introducido el módulo en el intérprete, podemos solicitar la evaluación de una expresión mediante el comando `reduce`:

```
Maude> reduce s(s(0)) + s(s(s(0))) .
result NzNat: s(s(s(s(s(0)))))
```

El intérprete aplica las ecuaciones como reglas de reescritura hasta alcanzar la forma normal e infiere el menor sort posible para el resultado, en este caso `NzNat`.

Para que un módulo funcional tenga buenas propiedades operacionales, las ecuaciones deben ser confluentes y operacionalmente terminantes, las mismas dos nociones que estudiábamos en el capítulo del λ-cálculo, ahora reformuladas en el contexto de un sistema de reescritura arbitrario. La terminación operacional garantiza que toda secuencia de reescritura concluye en un número finito de pasos, y la confluencia, que el resultado de la evaluación no depende del orden en que se contraen los redexes. Ambas propiedades, indecidibles en general, pueden chequearse automáticamente con las herramientas del *Maude Formal Environment* (Durán et al., 2020), entre las que destacan `MTT` para terminación y `ChC` para confluencia y suficiente completitud. Bajo estas condiciones, todo término se reduce a una forma normal única, lo que dota al lenguaje de una semántica determinista para sus módulos funcionales.

4.3. Módulos de Sistema y Reescritura no Determinista

Junto a los módulos funcionales, Maude provee módulos de sistema, declarados entre `mod` y `endm`. Un módulo de sistema extiende a uno funcional con un conjunto adicional de reglas de reescritura $l \to r$ que modelan transiciones de un sistema, semánticamente distintas de las ecuaciones (Clavel et al., 2007). Las ecuaciones describen *cómo se calcula el estado de un término*, esto es, cómo se simplifica a su forma normal. Las reglas, en cambio, describen *cómo evoluciona el sistema*, qué transiciones puede dar de un estado a otro. La diferencia es que las reglas no se asumen confluentes ni terminantes, sino que codifican el comportamiento intrínsecamente no determinista del sistema. Un mismo término puede reescribirse de maneras distintas y conducir a estados muy diferentes, y eso es justamente lo que se quiere capturar al describir, por ejemplo, las transiciones de un protocolo de comunicación, las jugadas de un juego, o los pasos de una máquina abstracta de evaluación para un lenguaje de programación.

La semántica operacional resultante combina ambos niveles. Ante un término, Maude normaliza primero con las ecuaciones y aplica después una regla al resultado. Para explorar el espacio de estados alcanzables desde un término inicial, Maude provee los comandos `rewrite`, que sigue una traza de reescrituras, y `search`, que explora todos los estados alcanzables en anchura buscando uno que satisfaga un patrón dado (Durán et al., 2020). Este último es de especial utilidad cuando se modela un sistema de tipos como un módulo de sistema: la búsqueda de derivaciones de tipado se reduce a una búsqueda en el árbol de reescrituras, y el comando devuelve, si la hay, la derivación que justifica el juicio.

4.4. Formalización de Lenguajes y Sistemas de Tipos

Los antecedentes de esta metodología se han recogido en chapter 3.

Para hacer visible este traslado en el caso más elemental, consideremos un fragmento del lambda cálculo sin tipos. Especificamos los términos con tres constructores que reflejan directamente la definición inductiva del capítulo anterior:

```
fmod LAMBDA-SYNTAX is
  protected STRING .
  sort Var .
  sort Term .
  subsort Var < Term .

  op var : String -> Var [ctor] .
  op lam : Var Term -> Term [ctor] .
  op app : Term Term -> Term [ctor] .
endfm
```

Aquí Var es subsort de Term, lo que captura sintácticamente el hecho de que toda variable es un término. Tanto la aplicación como la lambda abstracción, son constructres que, proporcionando los argumentos que exige su signatura será posible producir un sort. Aún así, Maude posee capacidades de expresividad más avanzadas, pues es posible sacar provecho de los operadores infix, permitiéndonos redefinir, por ejemplo, la aplicación como sigue:

```
  op _@_: Term Term -> Term [ctor] .
```

El underscore es un placeholder para los argumentos de la expresión. La β-reducción puede expresarse después como una regla de reescritura en un módulo de sistema que extienda al anterior. Para evitar la captura de variables se necesitaría una operación auxiliar de sustitución, definida por ecuaciones, que renombre las variables ligadas con identificadores frescos cuando sea necesario. Sin embargo, omitirémos esta funcionalidad por propósitos prácticos a lo largo de esta tesis para implementar el lenguaje dependientemente tipado y, posteriormente, su extensión con tipos cociente.

La fuerza de esta aproximación se aprecia mejor con un ejemplo menos trivial. Los términos de un sistema de tipos a la Church incluuyen, junto a las variables y aplicaciones, anotaciones de tipo en cada uno de sus binders, y las reglas de tipado se expresan en forma de juicios: $\Gamma \vdash M : A$. En Maude, basta con introducir un sort para los contextos, otro para los juicios de tipado, y reglas de reescritura que codifiquen cada regla de inferencia.

Tomando como ejemplo el contexto, el mismo puede definirse como un mapeo de variables a tipos, y tipos a tipos de variable $\mathsf{x} \to \tau$, $\mathsf{Nat} \to \mathcal{U}$. El contexto por tanto, almacena información sobre los tipos bien formados en nuestro programa. De no existir un tipo o término para un tipo en Γ, decimos que es un término o tipo malformado.

$$\Gamma := \quad \emptyset \tag{4.1}$$

$$\mid \quad (x, \tau) :: \Gamma \tag{4.2}$$

La noción de contexto puede traducirse en Maude, como hemos mencionado, creando un sort para este y las ecuaciones que le corresponden. Guiándonos, pues, de la gramática BNF, el contexto puede bien estar vacío, o extendido. Estas operaciones, pueden ser perfectamente codificadas de la siguiente forma:

```
fmod LAMBDA-SYNTAX is
  ...
  sort Ctx .
  op mt : -> Ctx [ctor] .
```

```
5   op _;_:_ : Ctx Var Term -> Ctx [ctor] .
6   var G : Ctx .
7 endfm
```

Donde la notación _;_:_, es un constructor para un sort de tipo *mt; var x : τ*, que representa el contexto vacío, extendido con *x : τ*.

Tomando como ejemplo la regla de formación del tipo sigma

$$\frac{\Gamma \vdash A : \mathsf{Type} \quad \Gamma, x : A \vdash B : \mathsf{Type}}{\Gamma \vdash \Sigma\, x : A.\, B : \mathsf{Type}}\ \Sigma\text{-Form} \tag{4.3}$$

Que puede leerse como: *si en el contexto Γ el término A es un tipo bien formado y, al extender dicho contexto con la hipótesis de que $x : A$, es posible inferir que también lo es B, entonces $\Sigma\, x : A.\, B$ es un tipo en Γ*. El equivalente en Maude de esta regla de formación, es la siguiente:

```
1 fmod LAMBDA-SYNTAX is
2   ...
3   op infer : Ctx Term -> [Term] .
4   ceq infer(G, Sigma(V, A, B)) = Type
5     if Type := norm(infer(G, A))
6     /\ Type := norm(infer(G ; V : A, B)) .
7 endfm
```

Obsérvese que su interpretación no es muy distinta a lo que la regla *(2.27)* formaliza: podemos inferir que $Sigma(V, A, B)$ en la ecuación, es un tipo si se cumple que A es un tipo en Γ y, en el mismo contexto extendido, V es de tipo A, B es también un tipo bien formado.

La ejecución del prototipo, pues, se convierte entonces en la búsqueda de derivaciones, y el comando `search` permite preguntar al sistema si existe una derivación con un consecuente dado, recuperando el término y la sustitución que la justifican (Durán et al., 2020; Clavel et al., 2007).

4.5. Reflexión y meta-programación

Un último aspecto que conviene mencionar, dada su utilidad cuando se construye un prototipo de un sistema de tipos, es que la lógica de reescritura es reflexiva (Clavel et al., 2007). Esto quiere decir que existe una teoría universal $\mathcal{U}$, finita y axiomatizable, en la que toda teoría de reescritura $\mathcal{R}$, incluida la propia $\mathcal{U}$, puede representarse como un término $\overline{\mathcal{R}}$, y los pasos de reescritura $t \to t'$ de $\mathcal{R}$ se corresponden con pasos de reescritura $\langle \overline{\mathcal{R}}, \overline{t} \rangle \to \langle \overline{\mathcal{R}}, \overline{t'} \rangle$ en $\mathcal{U}$. Maude implementa esta reflexión en el módulo predefinido `META-LEVEL`, que ofrece operaciones para manipular módulos, términos y reglas como datos de primera clase (Durán et al., 2020). Esto resulta de gran ayuda al implementar verificadores de tipos: el verificador puede inspeccionar el módulo que codifica el lenguaje, aplicar reglas de inferencia a términos meta-representados y devolver derivaciones como resultado, todo dentro de Maude y sin necesidad de salir al lenguaje anfitrión.

En definitiva, Maude provee una forma sumamente natural de implementar reglas

de inferencia y la semantica de un lenguaje de manera ecuacional y visual los pasos en los que se computa cada expresión que se suministre en pasos de reducción.

Capítulo 5

Un Lenguaje Dependientemente Tipado en Maude

El lenguaje que se propone en el presente trabajo, parte de un subconjunto del lambda cálculo simplemente tipado, según lo descrito por Löh et al. (2010) en su publicación sobre un lambda cálculo dependientemente tipado, donde nuestra implementación, pese a que difiere ligeramente de la suya, definirá las bases para el prototipo de lenguaje que en las próximas páginas estaremos formalizando. Asimismo, se ha elegido Maude como lenguaje base, por su simpleza para formalizar reglas de inferencia, como se ha podido demostrar. La sintáxis abstracta para términos y tipos, se define como sigue.

5.1. Sintaxis

$$
\begin{aligned}
e, \rho, k := \quad & \alpha && (1)\\
\mid \quad & e :: \rho && (2)\\
\mid \quad & * && (3)\\
\mid \quad & \forall x :: \rho . \rho' && (4)\\
\mid \quad & \lambda x.\, e && (5)\\
\mid \quad & e\ e' && (6)
\end{aligned}
$$

Nótese, como anteriormente se ha cubierto, los términos y los tipos comparten el mismo nivel semántico y no existe distinción entre ambos, por tanto, pudiese un término ocurrir en un tipo. Con respecto a (2), por motivos prácticos, nuestra implementación no manejará un nodo de anotación explícita, por lo que, en Maude, será totalmente seguro omitirlo.

```
fmod LAMBDA-QTT is
  ...
  sort Var .
  sort Term .
  subsort Var < Term .
  vars N N' : String .
  vars T T1 T1' T2 S A B : Term .
  var V : Var .
  op var : String -> Var [ctor] .
  op Type : -> Term [ctor] .
  op Pi : Var Term Term -> Term [ctor] .
  op lam : Var Term Term -> Term [ctor] .
  op app : Term Term -> Term [ctor] .
  ...
endfm
```

El segmento de constructor de tipos es de mayor interés. Es tentador definir un subsort de Term, tal que, dado un *Type* también podamos obtener un *Term* de forma intercambiable

```
  subsort Type < Term .
```

Pero esto es incorrecto, pues no tiene *Type* un orden estructuralmente menor que *Term*, por el contrario, dado que defininimos un cálculo dependiente, términos y tipos vienen siendo lo mismo semánticamente.

5.2. Sustitución

Las reglas de sustitución para nuestro lenguaje no son necesariamente distintas de las del lambda cálculo sin tipos discutidas en en un capitulo previo a este, aún así, existen diferencias que son notables en la sustitución del cálculo dependiente. Se pretende, por tanto, que las mismas sean triviales para el lector:

1. Las anotaciones de tipo de los binders para el tipo Π, es decir, A en $\lambda x : A.\, t$ y en $\Pi x : A.\, B$, son ahora términos y por tanto pueden contener ocurrencias libres de variables. La sustitución debe aplicarse de forma recursiva sobre el término.
2. Cuando se sustituye bajo un binder de variable y distinta de x, el cuerpo del binder se sustituye normalmente; cuando $y = x$, la sustitución se detiene en la anotación de tipo pero no avanza al cuerpo, pues x queda capturada por el binder.

La regla correspondiente a la sustitución en el tipo de función dependiente es omitida por Löh et al. (2010) sin embargo, es prudente mencionarla en nuestra implementación.

$$\Pi y : A.B[x := s] = \Pi y : A[x := s].B[x := s]$$

La regla es trivial para λ y Σ que describirémos posteriormente. Para el resto de casos, la sustitución se propaga estructuralmente.

Luego, la implementación en Maude define las reglas de cómputo para la sustitución en el cálculo dependientemente tipado, se define una regla por cada constructor

del cálculo, definiendo la condición de igualdad o desigualdad sobre los nombres de las variables, expresada de manera ecuacional:

```
fmod LAMBDA-QTT is
  ...
  op subst : Term Var Term -> Term .

  eq subst(var(N), var(N), S) = S .
  ceq subst(var(N), var(N'), S) = var(N) if N =/= N' .
  eq subst(app(T1, T2), V, S) = app(subst(T1, V, S), subst(T2, V, S)) .
  eq subst(Type, V, S) = Type .
  ceq subst(lam(var(N), A, T), var(N), S) = lam(var(N), subst(A, var(N), S), T)
    if true = true .
  ceq subst(lam(var(N), A, T), var(N'), S) =
    lam(var(N), subst(A, var(N'), S), subst(T, var(N'), S))
    if N =/= N' .
  ceq subst(Pi(var(N), A, B), var(N), S) = Pi(var(N), subst(A, var(N), S), B)
    if true = true .
  ceq subst(Pi(var(N), A, B), var(N'), S) =
    Pi(var(N), subst(A, var(N'), S), subst(B, var(N'), S))
    if N =/= N' .
  ...
endfm
```

5.3. Normalización

Se define, a continuación, la semántica operacional *big step* del proceso de normalización para cada término en el lenguaje, y con ella, la ecuación de Maude que corresponde a cada regla de evaluación.

$$\frac{}{\text{var}(x) \Downarrow \text{var}(x)}\ \text{Var} \qquad \texttt{eq norm(var(N)) = var(N) .}$$

$$\frac{}{\text{Type} \Downarrow \text{Type}}\ \text{Type} \qquad \texttt{eq norm(Type) = Type .}$$

$$\frac{A \Downarrow A' \quad B \Downarrow B'}{\text{Pi(V, A, B)}}\ \Pi \qquad \texttt{eq norm(Pi(V, A, B)) = Pi(V, norm(A), norm(B)) .}$$

$$\frac{A \Downarrow A' \quad T \Downarrow T'}{\text{lam(V, A, T)} \Downarrow \text{lam(V, A', T')}}\ \lambda$$

```
eq norm(lam(V, A, T)) =
lam(V, norm(A), norm(T)).
```

$$\frac{T_1 \Downarrow T_1' \quad T_2 \Downarrow T_2'}{\text{normApp(V, } T_1\text{, } T_2\text{)}}\ App \qquad \frac{\text{subst}(T, V, S) \Downarrow T'}{\text{normApp}(\text{lam}(V, A, T), S) \Downarrow T'}\ \beta$$

$$\frac{}{\text{normApp}(T_1, T_2) \Downarrow app(\text{T}_1, T_2)}\ App - otherwise$$

```
eq norm(app(T1, T2))
= normApp(norm(T1),
norm(T2)).
```

$$\frac{}{\text{Nat} \Downarrow \text{Nat}}\ Nat \qquad \texttt{eq norm(Nat) = Nat.}$$

$$\frac{}{\text{zero} \Downarrow \text{zero}}\ zero \qquad \texttt{eq norm(zero) = zero.}$$

$$\frac{T \Downarrow T'}{\text{suc(T)} \Downarrow \text{suc(T')}}\ suc \qquad \texttt{eq norm(suc(T)) = suc(norm(T)).}$$

$$\frac{T_1 \Downarrow T'}{\text{natrec}(A, T_1, T_2, \text{zero}) \Downarrow T'}\ \text{natrec-zero}$$

$$\frac{\text{app}(\text{app}(T_2, T), \text{natrec}(A, T_1, T_2, T)) \Downarrow T'}{\text{natrec}(A, T_1, T_2, \text{suc}(T)) \Downarrow T'}\ \text{natrec-suc}$$

$$\frac{A \Downarrow A' \quad T_1 \Downarrow T_1' \quad T_2 \Downarrow T_2' \quad T \Downarrow T'}{\text{natrec}(A, T_1, T_2, T) \Downarrow \text{natrec}(A', T_1', T_2', T')}\ \text{natrec-otherwise}$$

$$\frac{A \Downarrow \text{A'} \quad \text{B} \Downarrow \text{B'}}{\text{Sigma(V, A, B)} \Downarrow \text{Sigma(V, A', B')}}\ \Sigma \qquad \texttt{eq norm(Sigma(V, A, B)) = Sigma(V, norm(A), norm(B)).}$$

$$\frac{\text{T}_1 \Downarrow T_1' \quad T_2 \Downarrow T_2' \quad A \Downarrow \text{A'}}{pair(T_1, T_2, A) \Downarrow \text{pair}(T_1', T_2', A')}\ Pair \qquad \texttt{eq norm(pair(T1, T2, A)) = pair(norm(T1), norm(T2), norm(A)).}$$

$$\frac{\text{T}_1 \Downarrow T_1'}{\text{fst(pair}(T_1, T_2, \text{A)}) \Downarrow T_1'}\ fst \qquad \texttt{eq norm(fst(pair(T1, T2, A))) = norm(T1).}$$

$$\frac{\text{T}_1 \Downarrow T_1'}{\text{fst}(T_1) \Downarrow \text{fst}(T_1')}\ fst - otherwise \qquad \texttt{eq norm(fst(T)) = fst(norm(T)) [owise].}$$

$$\frac{\text{T}_2 \Downarrow T_2'}{\text{snd(pair}(T_1, T_2, \text{A)}) \Downarrow T_2'}\ snd \qquad \texttt{eq norm(fst(pair(T1, T2, A))) = norm(T2).}$$

$$\frac{\text{T} \Downarrow T'}{\text{snd}(T) \Downarrow \text{snd}(T')}\ snd - otherwise \qquad \texttt{eq norm(snd(T)) = snd(norm(T)) [owise].}$$

5.4. Verificación de Tipos

La verficación de tipos que se implementa es bidireccional. Se proponen las siguientes ecuaciones *check* (↓) e $infer$ (↑) para asertar que un término es de un tipo o respectivamente, sintetizar su tipo dado un contexto Γ cuando no hay suficiente información para asertar su estructura en un principio. Para propósitos prácticos, cada término en el lenguaje que se formaliza tiene una adscripción (es decir, está anotado con su signatura o el tipo en el que habita) y cualquier término no tipado será rechazado por el typechecker.

```
op check : Ctx Term Term -> [Bool] .        (5.1)
op infer : Ctx Term -> [Term] .             (5.2)
op tyEq : Term Term -> Bool .               (5.3)
```

$TyEq$ determina posteriormente si $T_1 =_\beta T_2$ luego de normalizar exhaustivamente, como consecuencia de la ocurrencia de términos en los tipos. La correspondencia entre las reglas de tipado y el código Maude está basada directamente en las del lambda cálculo dependientemente tipado sugeridas por Löh et al. (2010). Como observación, la implementación que se detalla diverge ligeramente en los constructores $Sigma$, fst y snd, pues no son propias del lambda dependientemente tipado que inicialmente Löh propone, pero jugarán un papel fundamental en los casos de uso que al momento de presentar los tipos cociente en nuestro sistema, se mostrarán.

$$\frac{}{\Gamma \vdash * \uparrow *}\ Type$$

```
eq infer(G, Type) = Type.
```

$$\frac{\Gamma \vdash A \downarrow * \quad \Gamma, x : A \vdash B \uparrow *}{\Gamma \vdash Pi(x, A, B) \uparrow *}\ Pi$$

```
ceq infer(G, Pi(V, A,
B)) = Type if Type :=
norm(infer(G, A)) /\ Type
:= norm(infer(G ; V : A,
B)) .
```

$$\frac{\Gamma \quad lookup(x) = \tau}{\Gamma \vdash x \uparrow \tau}\ Var$$

```
eq infer(G, var(N)) =
lookup(var(N), G).
```

$$\frac{\Gamma \vdash T_1 \uparrow Pi(x, A, B) \quad \Gamma \vdash T_2 \downarrow A}{\Gamma \vdash app(T_1, T_2) \uparrow B[x := T_2]}\ App$$

```
ceq infer(G, app(T1,
T2)) = norm(subst(B, V,
T2)) if Pi(V, A, B) :=
norm(infer(G, T1)) /\
true := check(G, T2, A) .
```

$$\frac{\Gamma \vdash A \uparrow * \quad \Gamma, x : A \vdash B \uparrow *}{\Gamma \vdash \lambda x : A.T \downarrow Pi(x, A, B)}\ Lam$$

```
ceq infer(G, lam(V, A,
T)) = Pi(V, A, B) if Type
:= norm(infer(G, A)) /\ B
:= infer(G ; V : A, T) .
```

$$\frac{}{\Gamma \vdash Nat \downarrow *}\ Nat$$

```
eq infer(G, Nat) = Type.
```

$$\frac{}{\Gamma \vdash zero \downarrow Nat}\ zero$$

```
eq infer(G, zero) = Nat .
```

$$\frac{\Gamma \vdash T \downarrow Nat}{\Gamma \vdash suc(T) \uparrow Nat}\ suc$$

```
ceq infer(G, suc(T)) =
Nat if true := check(G,
T, Nat).
```

$$\frac{\Gamma \vdash A \downarrow * \quad \Gamma, x : A \vdash B \uparrow *}{\Gamma \vdash Sigma(V, A, B) \uparrow *}\ \Sigma$$

```
ceq infer(G, Sigma(V,
A, B)) = Type if Type :=
norm(infer(G, A)) / Type
:= norm(infer(G ; V : A,
B)) .
```

$$\frac{\Gamma \vdash T_1 : A \quad \Gamma \vdash T_2 : B[x := T_1]}{\Gamma \vdash pair(T_1, T_2, Sigma(V, A, B)) \uparrow Sigma(V, A, B)}\ Pair$$

```
ceq infer(G, pair(T1,
T2, Sigma(V, A, B))) =
Sigma(V, A, B) if true
:= check(G, T1, A) /
true := check(G, T2,
norm(subst(B, V, T1)))
.
```

$$\frac{\Gamma \vdash T \uparrow Sigma(V, A, B)}{\Gamma \vdash fst(T) \uparrow A}\ fst$$

```
ceq infer(G, fst(T)) =
A if Sigma(V, A, B) :=
norm(infer(G, T)) .
```

$$\frac{\Gamma \vdash T \uparrow Sigma(V, A, B)}{\Gamma \vdash snd(T) \uparrow B[x := fst(T)]}\ snd$$

```
ceq infer(G, fst(T)) =
A if Sigma(V, A, B) :=
norm(infer(G, T)) .
```

Capítulo 6

Extensión con Tipos Cociente

El lambda cálculo dependientemente tipado implementado en el capítulo anterior será la base sobre la que extenderémos el sistema con tipos cociente, guiándonos de lo sugerido por Li (2014). Como en el capítulo precedente, se listan las reglas de formación, introducción, eliminación y computación, y a cada una de ellas se le hace corresponder la ecuación Maude que la realiza en el prototipo.

Antes de detallar las reglas, conviene precisar de qué forma se amplía la sintaxis del lenguaje. La gramática presentada en el capítulo anterior se extiende con tres nuevos constructores, que se corresponden con la formación del cociente, la formación de una clase de equivalencia y la eliminación por *lifting*, respectivamente:

$$
\begin{aligned}
e, \rho, k := \quad & \ldots && (*)\\
\mid \quad & \mathrm{qtype}(A, R) && (7)\\
\mid \quad & \mathrm{qclass}(t, \mathrm{qtype}(A, R)) && (8)\\
\mid \quad & \mathrm{qelim}(f, \mathit{resp}, q) && (9)
\end{aligned}
$$

Donde (7) construye el tipo cociente $A/\!\sim$ a partir de un tipo A y de una relación R; (8) construye la clase de equivalencia $[t]$ de un representante $t : A$, anotada con el tipo cociente al que pertenece; y (9) eleva una función f definida sobre representantes a una función definida sobre el cociente, exigiendo un testigo resp de que f respeta la relación. Su correspondencia en Maude, se indica como sigue:

```
1  op qtype : Term Term -> Term [ctor] .
2  op qclass : Term Term -> Term [ctor] .
3  op qelim : Term Term Term -> Term [ctor] .
```

La sustitución se propaga sobre cada constructor y recursivamente sobre los tipos presentes en cada uno de ellos. Una vez más, cualquier caso de captura de variables es evitable, pues ningún constructor introduce un binder propio sobre la variable que se sustituye:

```
1  eq subst(qtype(A, R), V, S) = qtype(subst(A, V, S), subst(R, V, S)) .
2  eq subst(qclass(A, R), V, S) = qclass(subst(A, V, S), subst(R, V, S)) .
3  eq subst(qelim(F, A, R), V, S) =
```

```
4    qelim(subst(F, V, S), subst(A, V, S), subst(R, V, S)) .
```

Es importante mencionar otra distinción con respecto a lo propuesto por Li (2014) y nuestra teoría, y es que en su trabajo propone que toda relación $\sim$ habite en $Prop$, esto es por la teoría de tipos de Martin Lof, en donde los cocientes se encuentran como extensión de la metateoría y existe una distinción entre $Type$ y $Prop$. Nuestro sistema de tipos se limita a habitar en un solo universo $*$. Esta decisión no compromete en su totalidad los resultados obtenidos.

6.1. Introducción

$$\frac{\Gamma \vdash t : A}{\Gamma \vdash \mathrm{qclass}(t, \mathrm{qtype}(A, R)) : \mathrm{qtype}(A, R)}\ \text{Q-Intro} \tag{6.1}$$

Según lo fijado por la regla, la ecuación en Maude aserta que el tipo de un representante $[t]$ en el cociente es de tipo $A/\sim$, si en principio es posible confirmar que en el mismo contexto Γ, t habita en A.

```
1  ceq infer(G, qclass(T, qtype(A, R))) = qtype(A, R)
2    if true := check(G, T, A) .
```

La decisión de anotar el tipo cociente sobre la clase responde al mismo principio que en el capítulo anterior abordamos con las tuplas del tipo Σ, donde pair posee su anotación. Sin dicha anotación, la síntesis del tipo de una clase sería ambigua, pues un mismo representante $t : A$ pertenece a tantos cocientes como relaciones puedan definirse sobre A. Portando la clase su tipo cociente, la inferencia permanece dirigida por la sintaxis y, por tanto, se mantiene decidible.

Puesto a que A y R son dentro de nuestro universo $*$ términos, es sensato normalizar ambos argumentos. De manera que la ecuación en Maude es congruente:

```
1  eq norm(qtype(A, R)) = qtype(norm(A), norm(R)) .
```

6.2. Formación

$$\frac{\Gamma \vdash A \uparrow *, \quad \Gamma a, b : A \vdash x \sim y \downarrow Pi(a, A, Pi(b, A, Type))}{\Gamma \vdash A/\sim \quad \uparrow *}\ \text{Q-Form} \tag{6.2}$$

La regla de formación sostiene que para inferir que $A/\sim$ es de tipo $*$, primero debe atestiguar que la relación de equivalencia entre a y b es transitiva, reflexiva y simétrica y, además, habitar en el tipo Π. Adicionalmente, a y b habitan A.

```
ceq infer(G, qtype(A, R)) = Type
if Type := norm(infer(G, A))
/\ true := check(G, R, Pi(var("a"), A, Pi(var("b"), A, Type))).
```

6.3. Eliminación y Computación

$$\frac{\Gamma \vdash Q \uparrow A/\sim \quad F \uparrow Pi(a, A, B)}{\Gamma \vdash qelim(F, Resp, Q) \uparrow B} \text{ Q-Elim} \tag{6.3}$$

Tanto la eliminación como la computación, juegan un papel importante en esta extensión a los cocientes pues, indican a Maude la forma en que deben reducirse las reglas del lenguaje. En esta regla en concreto, inferimos que *qelim* es de tipo B, asumiendo que F es una función cualquiera donde su $Resp$ es válida. Nótese que en nuestra implementación de Maude, agregamos como clausula la inferencia de que Q es de tipo $A/\sim$, es decir, A es un testigo bien definido bajo el cociente en Γ y es válido aplicar F sobre el.

```
ceq infer(G, qelim(F, Resp, Q)) = B
if qtype(A,R) := norm(infer(G, Q))
/\ Pi(V, A, B) := norm(infer(G, F)) .
```

La regla de computación describe cómo se comporta *lifting* en una función al aplicarse a una clase canónica. Nótese que $Q - Comp$ y $Q - Elim$ van de la mano, como se ha mencionado, y la razon es que la eliminación en principio describe las reglas para poder aplicar una función a un miembro del cociente y, por otro lado, la regla de cómputo describe, cómo de verdad obtener un valor a partir de ella. Nuestra implementación no tiene una regla concreta de $qcomp$, por lo que realmente basta y es mucho más práctico para nuestro propósito, cubrir directamente $qelim$ tomando en cuenta lo indicado por ambas reglas. Siguiendo pues, lo sugerido por Li (2014), al aplicar $\hat{f}$ a la clase $[a]$ de un representante a, se obtiene $f(a)$:

$$\frac{\mathrm{app}(F, T) \Downarrow V}{\mathrm{qelim}(F, A, \mathrm{qclass}(T, QT)) \Downarrow V} \text{ qelim-}\beta$$

$$\frac{F \Downarrow F' \quad A \Downarrow A' \quad R \Downarrow R'}{\mathrm{qelim}(F, A, R) \Downarrow \mathrm{qelim}(F', A', R')} \text{ qelim-otherwise}$$

En el prototipo de Maude, esta igualdad se realiza directamente como una regla de reescritura orientada de izquierda a derecha. Cuando el tercer argumento del eliminador es una clase canónica $\mathrm{qclass}(T, QT)$, el *lifting* se contrae aplicando F al representante T:

```
eq norm(qelim(F, A, qclass(T, QT))) = norm(app(F, T)) .
eq norm(qelim(F, A, R)) = qelim(norm(F), norm(A), norm(R)) [owise] .
```

La primera ecuación es la contraparte operacional de *Q-comp* que indica como se reduce $\hat{f}([t])$ a $f(t)$, donde la aplicación $\mathrm{app}(F, T)$ se normaliza a su vez mediante la β-reducción ya definida para el lenguaje base. La segunda ecuación, marcada con `owise`, constituye el caso cuando su tercer argumento no es una clase canónica, por ejemplo una variable, en este caso se normalizan los tres componentes y el eliminador permanece sin mutarse, porque no hay información suficiente sobre exactamente qué es ese tercer argumento, siendo imposible reducirlo por Maude.

A modo de ilustración, considérese el cociente de Nat por una relación R cualquiera y la elevación de la función sucesor. Tomando como función $\lambda x : \mathsf{Nat}.\, \mathrm{suc}(x)$ y como elemento la clase del representante $zero$, la síntesis del tipo y la computación proceden como sigue:

```
reduce in calc : norm(qelim(lam(var("x"), Nat, suc(var("x"))), var("resp"), qclass(zero, qtype
    (Nat, var("R"))))) .
rewrites: 12 in 0ms cpu (0ms real) (~ rewrites/second)
result Term: suc(zero)
Maude>
```

En este ejemplo, dado que tenémos una clase canónica $[zero]$ donde (asumimos que) R sí respeta la equivalencia, el tercer argumento de *qelim* tiene la forma *(T, R)*, es decir, un tipo bien formado y el cociente en el que está definida su relacíon de equivalencia, por tanto, la ecuación se reduce desenvolviendo la clase de equivalencia $[zero]$, aplicando f a su representante, en este caso $zero$, donde trivialmente obtenemos *suc(zero)* como resultado.

Es importante justificar una limitación, consecuencia de las decisiones tomadas para implementar el prototipo que, pese a que difieren del trabajo de Li (2014) no afecta de ninguna manera la simulación de los cocientes en nuestro lenguaje. Primero, la ausencia de la regla Qax, que se muestra:

$$\frac{\Gamma \vdash a, b : A \quad \Gamma \vdash p : a \sim b}{\Gamma \vdash Qax(p) : [a] =_{A/\sim} [b]}\ \text{Q-Ax} \tag{6.4}$$

La regla produce la igualdad proposicional en el cociente de $[a]$ y $[b]$. Internamente, nuestro lenguaje no implementa el tipo identidad que transporta pruebas de que dos elementos son iguales y, por tanto, enunciar que $[a]$ = $[b]$ cuando $a \sim b$, no es posible. Es por ello que esta igualdad es solo cierta en la metateoria del lenguaje y se asume de forma externa, es decir, asumimos que la relación es una equivalencia y las pruebas $Resp$ de que una f cualquiera respeta dicha relación para todos los elementos relacionados, es válida. Para apoyar este argumento, el tipo de $Resp$ es como sigue $Resp : (a, b : A) \to p : a \sim b \to f(a) = f(b)$, donde $f(a) = f(b)$ es un término que habita el tipo $Id(f(a), f(b))$, que es exactamente el testigo necesario para asertar esta igualdad.

Con las reglas de formación, introducción, eliminación y computación ya implementadas, el lenguaje dispone de tipos cociente en el sentido intensional de Li (2014). Los constructores fst, snd y el tipo Σ que se incorporaron en el capítulo anterior cobran ahora utilidad, pues permiten expresar los casos de uso característicos del cociente, como los enteros representados por pares de naturales o los multiconjuntos representados por listas módulo permutación, ejemplos que se desarrollan en chapter 7.

Capítulo 7

Casos de uso y validación

7.1. Los enteros como pares de naturales

7.2. Multiconjuntos como listas módulo permutación

7.3. Los enteros módulo n

7.4. Propiedades de la especificación

Capítulo 8

Conclusiones, limitaciones y trabajo futuro

8.1. Conclusiones

8.2. Limitaciones

8.3. Trabajo futuro

Apéndice A

Código Maude

```
fmod LAMBDA-QT is
  pr STRING .
  sort Var .
  sort Term .
  subsort Var < Term .

  vars N N' : String .
  vars T T1 T1' T2 S A F B R QT Resp Q : Term .
  var V : Var .

  op Nat : -> Term [ctor] .
  op zero : -> Term [ctor] .
  op suc : Term -> Term [ctor] .
  op natrec : Term Term Term Term -> Term [ctor] .
  op Sigma : Term Term Term -> Term [ctor] .
  op pair : Term Term Term -> Term [ctor] .
  op fst : Term -> Term [ctor] .
  op snd : Term -> Term [ctor] .

  op var : String -> Var [ctor] .
  op Type : -> Term [ctor] .
  op Pi : Var Term Term -> Term [ctor] .
  op lam : Var Term Term -> Term [ctor] .
  op app : Term Term -> Term [ctor] .

  op qtype : Term Term -> Term [ctor] .
  op qclass : Term Term -> Term [ctor] .
  op qelim : Term Term Term -> Term [ctor] .

  op subst : Term Var Term -> Term .

  eq subst(var(N), var(N), S) = S .
  ceq subst(var(N), var(N'), S) = var(N) if N =/= N' .
  eq subst(app(T1, T2), V, S) = app(subst(T1, V, S), subst(T2, V, S)) .
  eq subst(Type, V, S) = Type .
  ceq subst(lam(var(N), A, T), var(N), S) = lam(var(N), subst(A, var(N), S), T)
    if true = true .
  ceq subst(lam(var(N), A, T), var(N'), S) =
    lam(var(N), subst(A, var(N'), S), subst(T, var(N'), S))
    if N =/= N' .
  ceq subst(Pi(var(N), A, B), var(N), S) = Pi(var(N), subst(A, var(N), S), B)
    if true = true .
  ceq subst(Pi(var(N), A, B), var(N'), S) =
    Pi(var(N), subst(A, var(N'), S), subst(B, var(N'), S))
    if N =/= N' .
  eq subst(Nat, V, S) = Nat .
  eq subst(zero, V, S) = zero .
  eq subst(suc(T), V, S) = suc(subst(T, V, S)) .
  eq subst(natrec(T1, T2, A, B), V, S) =
```

```
    natrec(subst(T1, V, S), subst(T2, V, S), subst(A, V, S), subst(B, V, S)) .
ceq subst(Sigma(var(N), A, B), var(N), S) = Sigma(var(N), subst(A, var(N), S), B)
  if true = true .
ceq subst(Sigma(var(N), A, B), var(N'), S) =
  Sigma(var(N), subst(A, var(N'), S), subst(B, var(N'), S))
  if N =/= N' .
eq subst(pair(T1, T2, A), V, S) = pair(subst(T1, V, S), subst(T2, V, S), subst(A, V, S)) .
eq subst(fst(T), V, S) = fst(subst(T, V, S)) .
eq subst(snd(T), V, S) = snd(subst(T, V, S)) .
eq subst(qtype(A, R), V, S) = qtype(subst(A, V, S), subst(R, V, S)) .
eq subst(qclass(A, R), V, S) = qclass(subst(A, V, S), subst(R, V, S)) .
eq subst(qelim(F, A, R), V, S) = qelim(subst(F, V, S), subst(A, V, S), subst(R, V, S)) .

op norm : Term -> Term .
op normApp : Term Term -> Term .

eq norm(var(N)) = var(N) .
eq norm(Type) = Type .

eq norm(Pi(V, A, B)) = Pi(V, norm(A), norm(B)) .
eq norm(lam(V, A, T)) = lam(V, norm(A), norm(T)) .

eq norm(app(T1, T2)) = normApp(norm(T1), norm(T2)) .

eq normApp(lam(V, A, T), S) = norm(subst(T, V, S)) .

eq normApp(T1, T2) = app(T1, T2) [owise] .

eq norm(Nat) = Nat .
eq norm(zero) = zero .

eq norm(suc(T)) = suc(norm(T)) .

eq norm(natrec(A, T1, T2, zero)) = norm(T1) .

eq norm(natrec(A, T1, T2, suc(T))) =
  norm(app(app(T2, T), natrec(A, T1, T2, T))) .

eq norm(natrec(A, T1, T2, T)) =
  natrec(norm(A), norm(T1), norm(T2), norm(T)) [owise] .

eq norm(Sigma(V, A, B)) = Sigma(V, norm(A), norm(B)) .

eq norm(pair(T1, T2, A)) = pair(norm(T1), norm(T2), norm(A)) .

eq norm(fst(pair(T1, T2, A))) = norm(T1) .

eq norm(fst(T)) = fst(norm(T)) [owise] .

eq norm(snd(pair(T1, T2, A))) = norm(T2) .

eq norm(snd(T)) = snd(norm(T)) [owise] .

eq norm(qtype(A, R)) = qtype(norm(A), norm(R)) .
eq norm(qclass(A, R)) = qclass(norm(A), norm(R)) .
eq norm(qelim(F, A, qclass(T, QT))) = norm(app(F, T)) .
eq norm(qelim(F, A, R)) = qelim(norm(F), norm(A), norm(R)) [owise] .

op tyEq : Term Term -> Bool .
eq tyEq(A, B) = norm(A) == norm(B) .

sort Ctx .
op mt : -> Ctx [ctor] .
op _`;_`:_ : Ctx Var Term -> Ctx [ctor] .
var G : Ctx .

op lookup : Var Ctx -> [Term] .
eq lookup(var(N), G ; var(N) : A) = A .
ceq lookup(var(N), G ; var(N') : A) = lookup(var(N), G) if N =/= N' .

```

```
120   op infer : Ctx Term -> [Term] .
121   op check : Ctx Term Term -> [Bool] .
122
123   eq infer(G, Type) = Type .
124
125   eq infer(G, var(N)) = lookup(var(N), G) .
126
127   ceq infer(G, Pi(V, A, B)) = Type
128     if Type := norm(infer(G, A))
129     /\ Type := norm(infer(G ; V : A, B)) .
130
131   ceq infer(G, lam(V, A, T)) = Pi(V, A, B)
132     if Type := norm(infer(G, A))
133     /\ B := infer(G ; V : A, T) .
134
135   ceq infer(G, app(T1, T2)) = norm(subst(B, V, T2))
136     if Pi(V, A, B) := norm(infer(G, T1))
137     /\ true := check(G, T2, A) .
138
139   eq infer(G, Nat) = Type .
140
141   eq infer(G, zero) = Nat .
142
143   ceq infer(G, qtype(A, R)) = Type
144   if Type := norm(infer(G, A))
145   /\ true := check(G, R, Pi(var("a"), A, Pi(var("b"), A, Type))).
146
147   ceq infer(G, qclass(T, qtype(A, R))) = qtype(A, R)
148     if true := check(G, T, A) .
149
150   ceq infer(G, qelim(F, Resp, Q)) = B
151   if qtype(A,R) := norm(infer(G, Q))
152   /\ Pi(V, A, B) := norm(infer(G, F)) .
153
154   ceq infer(G, suc(T)) = Nat if true := check(G, T, Nat) .
155
156   ceq infer(G, natrec(A, T1, T2, T)) = norm(app(A, T))
157     if true := check(G, T, Nat) .
158
159   ceq infer(G, Sigma(V, A, B)) = Type
160     if Type := norm(infer(G, A))
161     /\ Type := norm(infer(G ; V : A, B)) .
162
163   ceq infer(G, pair(T1, T2, Sigma(V, A, B))) = Sigma(V, A, B)
164     if true := check(G, T1, A)
165     /\ true := check(G, T2, norm(subst(B, V, T1))) .
166
167   ceq infer(G, fst(T)) = A
168     if Sigma(V, A, B) := norm(infer(G, T)) .
169
170   ceq infer(G, snd(T)) = norm(subst(B, V, fst(T)))
171    if Sigma(V, A, B) := norm(infer(G, T)) .
172
173   eq check(G, T, A) = tyEq(infer(G, T), A) .
174 endfm
```

Bibliografía

Nipkov Baader. *Term Rewriting and All That*. Cambridge University Press, 2012.

Henk Barendregt, Wil Dekkers, and Richard Statman. *Lambda Calculus with Types*. Perspectives in Logic. Cambridge University Press, 2013.

Henk (Hendrik) Barendregt. An introduction to generalized type systems. *Journal of Functional Programming*, 1:125–154, 04 1991. doi: 10.1017/S0956796800020025.

Henk P. Barendregt. *The Lambda Calculus: Its Syntax and Semantics*, volume 103 of *Studies in Logic and the Foundations of Mathematics*. North-Holland, Amsterdam, revised edition, 1984.

Gilles Barthe and Thierry Coquand. An Introduction to Dependent Type Theory. In Gilles Barthe, Peter Dybjer, Luís Pinto, and João Saraiva, editors, *Applied Semantics*, volume 2395, pages 1–41. Springer Berlin Heidelberg. ISBN 978-3-540-44044-4 978-3-540-45699-5. doi: 10.1007/3-540-45699-6_1. URL `http://link.springer.com/10.1007/3-540-45699-6_1`.

Manuel Clavel, Francisco Durán, Steven Eker, José Meseguer, Patrick Lincoln, Narciso Martí-Oliet, and Carolyn Talcott. *All About Maude – A High-Performance Logical Framework*, volume 4350 of *Lecture Notes in Computer Science*. Springer, 2007.

Thierry Coquand. Type Theory. In Edward N. Zalta and Uri Nodelman, editors, *The Stanford Encyclopedia of Philosophy*. Metaphysics Research Lab, Stanford University, Fall 2022 edition, 2022.

Francisco Durán, Steven Eker, Santiago Escobar, Narciso Martí-Oliet, José Meseguer, Rubén Rubio, and Carolyn Talcott. Programming and symbolic computation in Maude. *Journal of Logical and Algebraic Methods in Programming*, 110:100497, 2020. doi: 10.1016/j.jlamp.2019.100497.

German Geuvers. *Type Theory and Formal Proof*. Cambride University Press, 2014.

Martin Hofmann. *Extensional Concepts in Intensional Type Theory*. PhD thesis, University of Edinburgh, 1995.

Martin Hofmann. *Syntax and Semantics of Dependent Types*. Cambridge University Press, 2009.

Nuo Li. *Quotient Types in Type Theory*. PhD thesis, University of Nottingham, September 2014.

Andres Löh, Conor McBride, and Wouter Swierstra. A tutorial implementation of a dependently typed lambda calculus. *Fundamenta Informaticae*, 102(2):177–207, 2010.

María Manzano and Victor Aranda. Many-sorted logic. The Stanford Encyclopedia of Philosophy, 2022. URL https://plato.stanford.edu/entries/logic-many-sorted/.

Per Martin-Löf. An intuitionistic theory of types: Predicative part. In H. E. Rose and J. C. Shepherdson, editors, *Logic Colloquium '73, Proceedings of the Logic Colloquium*, volume 80 of *Studies in Logic and the Foundations of Mathematics*, pages 73–118. Elsevier, 1975. doi: 10.1016/S0049-237X(08)71945-1.

Samuel Mimram. *Program = Proof*. Independently published, 2020.

Ulf Norell. *Towards a Practical Programming Language Based on Dependent Type Theory*. PhD thesis, Chalmers University of Technology, 2007.

Benjamin C. Pierce. *Types and Programming Languages*. MIT Press, 2002.

John C. Reynolds. The meaning of types: From intrinsic to extrinsic semantics. Technical Report RS-00-32, BRICS, Department of Computer Science, University of Aarhus, December 2000. URL https://www.cs.cmu.edu/afs/cs/user/jcr/ftp/typemeaning.pdf.